



Ministry of Higher Education
And Scientific Research



Final Year Project Report

Major:

Bachelor's degree in Big Data and Data Analysis

Directed by:

Azzabi Mohamed Soltan

Supervised by:

Academic Supervisor: Mr. Imed Ben Boukhatem

Professional Supervisor: Mr. Iheb Turki

Topic:

**Design and Development of a Web Application for Security Agent
Management at STB**



Dedication

To my parents,

for their constant support, patience, and unconditional love throughout my academic journey.

To my friends and classmates,

with whom I shared both the challenges and the achievements of these years of study.

This work is dedicated to you.

Acknowledgements

The completion of this final year project would not have been possible without the contribution of several people to whom I wish to express my sincere gratitude.

First and foremost, I would like to thank my academic supervisor, Mr. Imed Ben Boukhatem, for his availability, valuable guidance, and rigorous follow-up throughout the project. His advice and feedback were of great importance in steering this work.

I would also like to thank the entire staff of the Société Tunisienne de Banque (STB) who welcomed me warmly and facilitated access to the information necessary for the completion of this project. Their cooperation and professionalism directly contributed to the progress of this work.

My sincere thanks also go to all the faculty members of the Faculty of Information and Communication Technologies at Université Centrale de Tunis, for the quality of education provided throughout the three years of the bachelor's program.

Finally, I thank my family and friends for their moral support and constant encouragement.

Abstract

This report presents the work carried out during a four-month internship at the Société Tunisienne de Banque (STB), from February 1 to May 31, 2026. The main objective of this project is the design and development of a web application dedicated to the centralized and secure management of security agents at STB.

The application provides role-adapted interfaces for three user types: Administrators who manage agents, sites, assignments, and users; Team Leaders who track attendance, manage their teams, and submit incident reports; and Security Agents who access their profiles, assignments, and attendance history. A Big Data analytics module enables administrators to visualize aggregated performance indicators through PostgreSQL materialized views and an Apache Spark distributed processing prototype.

Security is enforced through JWT-based authentication and three-level hierarchical role-based access control. The backend is built with Node.js and Express 5, the database with PostgreSQL 16, and the frontend with Angular 19 using standalone components.

Keywords: Security agent management, Web application, Node.js, Angular 19, PostgreSQL, JWT, REST API, Big Data, Apache Spark, Materialized Views, RBAC, STB.

Resume

This report covers the work completed during a last stage of research conducted at the Tunisian Bank (STB) from May 1 to May 31, 2026. The primary objective is to design and develop a web application for managing security agents. The programme offers interfaces tailored to three roles (Administrateur, Chef d'Équipe, Agent), integrates a Big Data module with PostgreSQL and Apache Spark prototypes, and secures access via JWT and three levels of hierarchical access control.

Mots-clés : Gestion des agents de sécurité, Application web, Node.js, Angular 19, PostgreSQL, JWT, REST API, Big Data, Apache Spark, Vues matérialisées, STB.

Table of Contents

Dedication

Acknowledgements

Abstract

Resume

Table of Contents

List of Figures

List of Tables

List of Abbreviations

General Introduction

Chapter 1: Project Context and Host Organization

1.1 Presentation of the Host Organization

1.1.1 History and Overview

1.1.2 Fact Sheet

1.1.3 Activities and Services

1.2 Problem Statement

1.3 Project Objectives

Chapter 2: State of the Art

2.1 Web Application Architectures

2.1.1 Definition and Evolution

2.1.2 The Three-Tier Architecture

2.1.3 REST APIs

2.1.4 Single Page Applications (SPA)

2.2 Security Mechanisms

2.2.1 Authentication vs. Authorization

2.2.2 Session-Based vs. Token-Based Authentication

2.2.3 JSON Web Tokens (JWT)

2.2.4 Role-Based Access Control (RBAC)

2.2.5 Password Security: bcrypt

2.3 Big Data Technologies

2.3.1 Definition and the 5 V's of Big Data

2.3.2 OLAP and Materialized Views

2.3.3 Apache Spark

2.4 Development Methodology: Comparative Analysis and Selection

2.4.1 Waterfall Model

2.4.2 Agile Methodology

2.4.3 Scrum Framework

2.4.4 Methodology Comparison

2.4.5 Selected Methodology: Agile-Inspired Iterative Approach

2.5 Technology Stack Justification

2.5.1 Backend Framework

2.5.2 Frontend Framework

2.5.3 Database

Chapter 3: Requirements Analysis

3.1 System Actors

3.1.1 Administrator (Level 3 — Full Access)

3.1.2 Team Leader / Chef d'Équipe (Level 2 — Intermediate)

3.1.3 Security Agent (Level 1 — Restricted)

3.2 Functional Requirements

3.3 Non-Functional Requirements

3.4 Use Case Diagrams

3.5 Product Backlog

Chapter 4: System Design

4.1 Overall System Architecture

4.2 Data Model

4.2.1 Entity Descriptions

4.2.2 Entity-Relationship Diagram

4.2.3 Key Relationships

4.3 Role Permissions Matrix

4.4 Sequence Diagrams

4.4.1 JWT Authentication

4.4.2 Agent Creation with Auto-Assignment

4.4.3 Two-Step Request Approval

Chapter 5: Implementation and Testing

5.1 Development Environment

5.2 Backend Implementation

5.2.1 Project Structure

5.2.2 REST API Endpoints

5.3 Frontend Implementation

5.3.1 Angular 21 Standalone Architecture

5.3.2 Application Screens

5.4 Security Implementation

5.4.1 Password Hashing with bcrypt

5.4.2 JWT Authentication

5.4.3 CORS Configuration

5.5 Big Data Analytics Module

5.5.1 Materialized Views (Production)

5.5.2 Apache Spark ETL Prototype

General Conclusion and Perspectives

Future Perspectives

Bibliography

List of Figures

Figure 1.1 – STB Organizational Chart	6
Figure 2.1 – Three-Tier Architecture Overview	14
Figure 2.2 – REST API Communication Model	16
Figure 2.3 – JWT Authentication Flow Diagram	18
Figure 2.4 – Scrum Sprint Cycle	23
Figure 3.1 – Overall Use Case Diagram – STB Security Management System	31
Figure 3.2 – Use Case Diagram – Administrator	32
Figure 3.3 – Use Case Diagram – Team Leader	33
Figure 3.4 – Use Case Diagram – Security Agent	33
Figure 4.1 – System Architecture (Full 3-Tier)	40
Figure 4.2 – Entity-Relationship Diagram (7 Entities)	44
Figure 4.3 – Sequence Diagram – JWT Authentication	46
Figure 4.4 – Sequence Diagram – Agent Creation with Auto-Assignment	48
Figure 4.5 – Sequence Diagram – Two-Step Request Approval Workflow	49
Figure 5.1 – Login Page	55
Figure 5.2 – Administrator Dashboard	56
Figure 5.3 – Team Leader Dashboard	56
Figure 5.4 – Security Agent Dashboard	57
Figure 5.5 – Agent List Page	57
Figure 5.6 – Add Agent Form	58
Figure 5.7 – Bank Sites with Interactive Leaflet Map	58
Figure 5.8 – Assignments Management	59
Figure 5.9 – Attendance Recording (Pointage)	59
Figure 5.10 – Big Data Analytics Dashboard	60
Figure 5.11 – API Testing with Postman	62

List of Tables

Table 1.1 – List of Abbreviations Table	
Table 1.2 – STB Fact Sheet	5
Table 2.1 – Methodology Comparison: Waterfall vs. Agile vs. Scrum	24
Table 2.2 – Backend Framework Comparison	27
Table 2.3 – Frontend Framework Comparison	28
Table 2.4 – Database Comparison	29
Table 3.1 – Functional Requirements	34
Table 3.2 – Non-Functional Requirements	36
Table 3.3 – Product Backlog (with story points)	
Table 3.4 – Sprint Backlog for Sprint 1	
Table 3.5 – Sprint Backlog for Sprint 2	
Table 3.6 – Sprint Backlog for Sprint 3	
Table 3.7 – Sprint Backlog for Sprint 4	
Table 3.8 – Sprint Backlog for Sprint 5	
Table 3.9 – Sprint Backlog of Sprint 6	
Table 3.10 – Sprint Backlog of Sprint 7	
Table 4.1 – Database Entities Description	43
Table 4.2 – Role Permissions Matrix	45
Table 5.1 – Development Environment	53
Table 5.2 – REST API Endpoints Summary	61
Table 5.3 – Functional Test Results	63

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
HTTP / HTTPS	HyperText Transfer Protocol / Secure
CRUD	Create, Read, Update, Delete
STB	Société Tunisienne de Banque
PFE	Projet de Fin d'Études (Final Year Project)
ERD	Entity-Relationship Diagram
SPA	Single Page Application
CORS	Cross-Origin Resource Sharing
RBAC	Role-Based Access Control
OLAP	Online Analytical Processing
ETL	Extract, Transform, Load
RDD	Resilient Distributed Dataset (Apache Spark)
MVC	Model-View-Controller
SSL/TLS	Secure Sockets Layer / Transport Layer Security
UI / UX	User Interface / User Experience
JSON	JavaScript Object Notation
LTS	Long-Term Support
ORM	Object-Relational Mapping
MV	Materialized View
JDBC	Java Database Connectivity
HMAC	Hash-based Message Authentication Code
KPI	Key Performance Indicator

Table 1.1 – List of Abbreviations Table

General Introduction

In an increasingly digital world, financial institutions face growing challenges in internal management, operational efficiency, and physical security. The Société Tunisienne de Banque (STB), one of Tunisia's largest public banks, operates an extensive network of over 130 branches spread across the national territory. The protection of these branches is carried out by a body of security agents whose management represents a significant operational challenge.

Before this project, the management of security agents at STB relied entirely on manual processes: Excel spreadsheets, paper registers, and informal communication channels. This fragmented approach made real-time monitoring of assignments impossible, introduced high risks of data errors, and prevented quick decision-making in urgent situations. There was no centralized system, no access control, and no analytical capability to derive insights from the accumulated data.

This final year project, carried out within the framework of the Bachelor's Degree in Big Data at Université Centrale de Tunis, aims to address these deficiencies by designing and developing a complete web application for the centralized and secure management of STB's security agents. The system covers agent profile management, bank site management, assignment tracking, daily attendance recording, incident reporting, multi-level request approval workflows, and a Big Data analytics module — all protected by JWT authentication and hierarchical role-based access control.

The application is built on a three-tier architecture: a RESTful backend with Node.js and Express 5, a PostgreSQL 16 relational database with materialized views, and a dynamic Angular 21 frontend. . A distributed processing prototype using Apache Spark demonstrates the Big Data competencies of the program.

This report is organized into five chapters:

- Chapter 1 – Project Context: introduces the host organization STB, the problem statement, and objectives.
- Chapter 2 – State of the Art: surveys key technologies, architectural patterns, and compares development methodologies with a justified final selection.
- Chapter 3 – Requirements Analysis: actors, functional and non-functional requirements, use case diagrams, and product backlog.
- Chapter 4 – System Design: three-tier architecture, data model (7 entities), permissions matrix, and sequence diagrams.
- Chapter 5 – Implementation and Testing: development environment, backend API, Angular frontend, security layer, Big Data module, and test results.

The report concludes with a general conclusion summarizing accomplished work and future improvement perspectives.

Chapter 1: Project Context and Host Organization

This chapter introduces the general framework of the final year project. We begin with a presentation of the host organization STB, before describing the problem statement, the project objectives, and the scope of work.

1.1 Presentation of the Host Organization

1.1.1 History and Overview

The Société Tunisienne de Banque (STB) is a Tunisian public commercial bank founded in 1957, shortly after Tunisia's independence. Created to support national economic development, the STB has grown into one of the country's most important financial institutions, serving a wide range of customers from individuals to large corporations and public institutions.

Today, the STB operates a network of over 130 branches across Tunisia, making it one of the densest banking networks in the country. It is listed on the Tunis Stock Exchange (BVMT) and plays a central role in financing both public and private sector projects.

1.1.2 Fact Sheet

Field	Detail
Corporate Name	Société Tunisienne de Banque (STB)
Legal Form	Public Limited Company – Public Bank
Year of Establishment	1957
Headquarters	Rue Hédi Nouira, Tunis, Tunisia
Share Capital	Over 300 million TND
Sector	Banking and Financial Services
Branch Network	Over 130 national branches
Stock Exchange Listing	Listed on the Tunis Stock Exchange (BVMT)
Website	www.stb.com.tn

Table 1.2 – STB Fact Sheet



STB Organizational Chart

1.1.3 Activities and Services

The STB provides a comprehensive range of banking products and services:

- Retail banking: current accounts, savings accounts, consumer and mortgage loans, payment cards.
- Corporate banking: investment loans, project financing, credit lines, international trade services.
- Digital services: online banking portal, mobile applications, electronic payments.
- Para-banking: insurance, leasing, wealth management.

Within its operations, the STB places particular importance on the physical security of its premises. Security agents are deployed across branches and headquarters to protect assets, employees, and customers — managing these agents efficiently is the core problem this project addresses.

1.2 Problem Statement

The management of security agents at STB represented a recurring and complex administrative task relying entirely on manual methods. Several concrete problems were identified:

- No centralized system to consult the real-time status of agent assignments across sites.
- Difficulty in quickly identifying available or unassigned agents in urgent situations.
- High risk of data entry errors due to unsynchronized Excel files with no shared access.
- No access control: anyone with file access could modify data without any audit trail.
- Time-consuming verification processes for managers before each shift.
- Impossibility of generating aggregated performance analytics on security resources.

These limitations created operational inefficiencies and data integrity risks that could not be tolerated in a banking environment. STB therefore expressed the need for a modern, secure, and centralized information system.

1.2.1 Existing Solutions and Why a Custom System Was Chosen

Commercial HR management systems (Oracle HCM, SAP SuccessFactors, Sage HR) exist but are not designed for security agent shift-based operations: they lack site-based RBAC hierarchies, two-step approval workflows tied to physical locations, and integrated Big Data analytics layers. Adapting such a system to STB's specific workflows would require costly licensed customization and vendor lock-in. Generic workforce management tools equally lack the domain-specific business rules identified (one team leader per site, 90-day auto-assignment, dual FK auditability in rapports). A bespoke system allows precise alignment with STB's operational model while remaining fully maintainable by the internal IT department.

1.3 Project Objectives

The main objective of this project is to design and develop a web application that digitizes and centralizes STB's security agent management. Specific objectives include:

- Full CRUD management of security agent profiles.
- Bank site management with team leader assignment (one leader per site).
- Agent-to-site assignment management with date ranges.
- Daily attendance tracking with status recording and time logs.
- Incident report submission and two-step validation workflow.
- JWT-based authentication with three-level hierarchical role-based access control.
- Big Data analytics module with PostgreSQL materialized views and Apache Spark prototype.
- Data security, integrity, and auditability.

This chapter situated the project in its organizational and functional context, identifying STB, the problem, and the project objectives. The following chapter surveys the key technologies and methodologies underpinning this work.

Chapter 2: State of the Art

Before designing any software system, it is essential to survey the existing landscape of technologies, architectural patterns, and development approaches. This chapter examines the foundational concepts underlying the STB Security application and closes with a fully justified methodology and technology stack selection.

2.1 Web Application Architectures

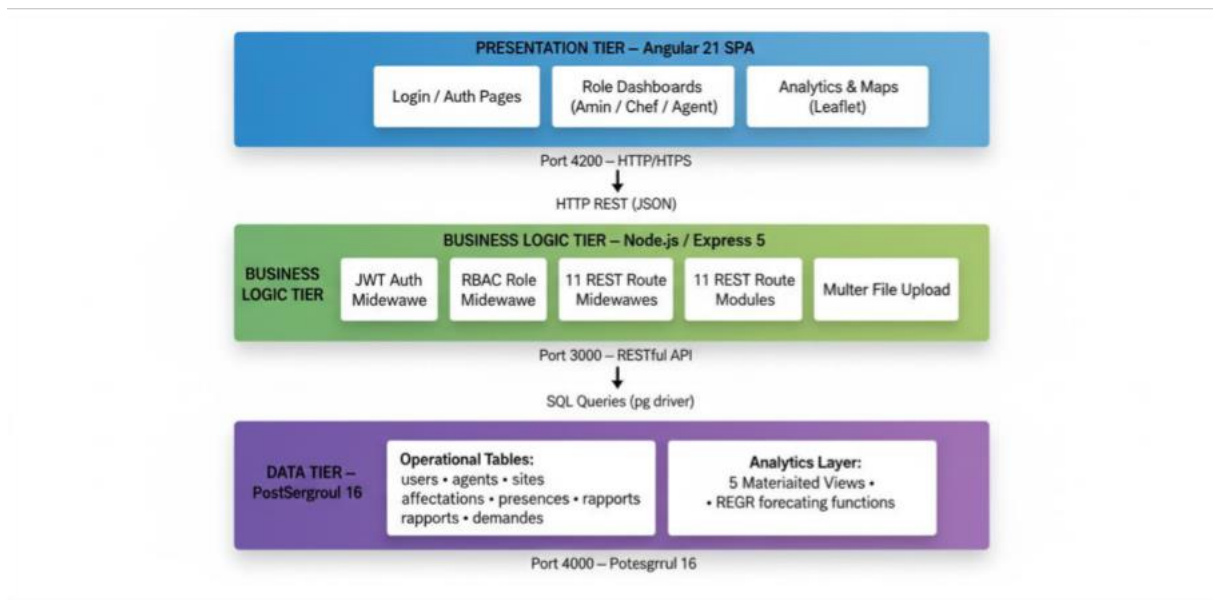
2.1.1 Definition and Evolution

A web application is a software system accessible via a web browser over a network, combining a user interface layer, a business logic layer, and a data persistence layer. Web applications have evolved significantly over three decades, from static HTML pages to complex distributed real-time systems. Early monolithic applications bundled all layers into a single deployable unit — simple initially but increasingly difficult to maintain and scale.

2.1.2 The Three-Tier Architecture

The three-tier architecture separates the system into three independent layers: Presentation (frontend), Business Logic (backend), and Database. This separation yields major advantages: independent scalability, cleaner codebases, easier testing, and team autonomy across layers.

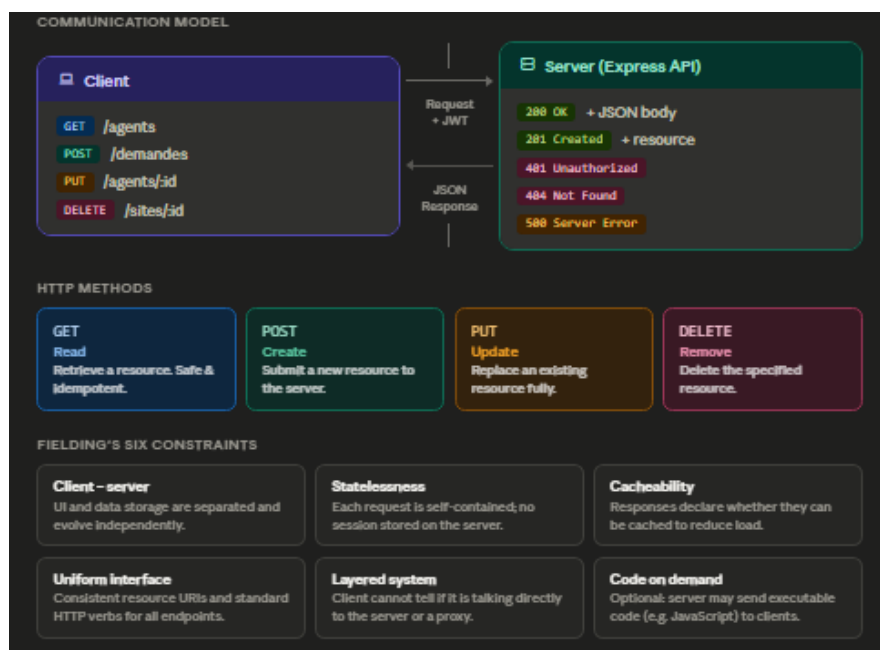
- **Presentation Tier:** user interface, built with Angular, React, or Vue. Communicates via HTTP.
 - **Business Tier:** Business rules, authentication, authorization. Node.js/Express, Django, Spring Boot.
 - **Data Tier:** persistent storage. Relational (PostgreSQL, MySQL) or NoSQL (MongoDB, Cassandra).
-



Three-Tier Architecture Overview

2.1.3 REST APIs

REST (Representational State Transfer), introduced by Roy Fielding in 2000, is an architectural style for networked applications. A RESTful API exposes resources via standard HTTP methods (GET, POST, PUT, DELETE) with stateless communication — each request must be self-contained. REST's six constraints (client-server, statelessness, cacheability, uniform interface, layered system, code-on-demand) enable scalable, interoperable, independently deployable services. JSON is the universal data format, given its lightweight syntax and universal library support.



REST API Communication Model

2.1.4 Single Page Applications (SPA)

A Single Page Application (SPA) loads a single HTML page and dynamically updates content without full page reloads. SPAs offer superior user experience: faster interactions, smooth navigation, and partial offline capability. Angular (Google, TypeScript-based), React (Meta), and Vue.js are the leading SPA frameworks.

2.2 Security Mechanisms

2.2.1 Authentication vs. Authorization

Authentication verifies identity ("Who are you?"); authorization determines permissions ("What can you do?"). Both are essential in any multi-user system handling sensitive data.

2.2.2 Session-Based vs. Token-Based Authentication

Traditional session-based authentication stores session state on the server — problematic for horizontal scaling and adding a database lookup cost to every request. Token-based authentication with JWT eliminates server-side state: the server generates a signed token on login and validates its signature on every subsequent request without any database query, making it inherently stateless and horizontally scalable.

2.2.3 JSON Web Tokens (JWT)

A JWT consists of three Base64URL-encoded parts: Header (algorithm and token type), Payload (claims: user ID, email, role, expiration), and Signature (HMAC-SHA256 of header+payload using the secret key). Tampering with any part invalidates the signature, making JWTs cryptographically secure. The expiration claim (exp) ensures tokens become invalid after a set period.



JWT Authentication Flow Diagram

2.2.4 Role-Based Access Control (RBAC)

RBAC associates permissions with roles rather than individual users. Users are assigned roles, and roles carry operation permissions. Hierarchical RBAC allows higher roles to inherit lower role permissions. This project implements three levels: Agent (1), Team Leader (2), Administrator (3).

2.2.5 Password Security: bcrypt

bcrypt is an adaptive password hashing function that incorporates a random salt and a configurable work factor, making brute-force attacks computationally expensive. It is the industry standard for secure password storage in web applications.

2.3 Big Data Technologies

2.3.1 Definition and the 5 V's of Big Data

Big Data refers to datasets too large or complex for traditional database systems. It is characterized by five dimensions:

- Volume: massive data quantities (terabytes to petabytes).
- Velocity: high speed of data generation and processing needs.
- Variety: multiple formats (structured, semi-structured, unstructured).
- Veracity: uncertainty and quality of data.
- Value: actionable business insights derivable from analysis.

2.3.2 OLAP and Materialized Views

OLAP (Online Analytical Processing) optimizes databases for complex analytical queries over large datasets. Materialized views are precomputed query results stored as physical database objects, refreshed periodically. Instead of re-executing expensive aggregations on every request, the database serves the precomputed snapshot — dramatically reducing analytical query latency. This approach is ideal for dashboards with frequent reads but infrequent data changes.

2.3.3 Apache Spark

Apache Spark is an open-source distributed computing engine for large-scale data processing. Its Resilient Distributed Dataset (RDD) abstraction enables fault-tolerant parallel processing across commodity hardware clusters. Spark's DataFrame API achieves in-memory computation speeds up to 100x faster than Hadoop MapReduce for certain workloads. PySpark (the Python API) provides a high-level interface for ETL pipelines using window functions, groupBy, and aggregations over distributed data.

2.3.4 Big Data Justification for the STB Security Project

A jury reviewing a Big Data specialization project will rightfully ask: why does this problem need Big Data? STB has 130+ branches. Each branch employs between 5 and 15 security agents. With daily attendance records per agent, the operational database accumulates approximately $130 \text{ branches} \times 10 \text{ agents average} \times 365 \text{ days} = 474,500$ presence records per year. Over five years, that exceeds 2.3 million rows in the presences table alone, before

including assignments, reports, requests, and audit logs. This is the operational baseline; at scale with the full national workforce, the volume justifies distributed processing.

To validate the analytics pipeline under realistic volume, the `generateBigData.js` script seeds 200 agents with two full years of daily attendance data, producing approximately 146,000 synthetic presence records. This simulates a medium-sized STB regional network and confirms that the materialized views and PySpark ETL execute correctly at this scale without query degradation.

The architectural decision to build on Apache Spark rather than plain SQL aggregation is motivated by three factors: (1) Scalability—the PySpark ETL script runs unchanged on a Hadoop/Spark cluster when the dataset outgrows a single PostgreSQL instance; (2) Columnar storage—Parquet output enables future machine learning workloads (absenteeism prediction, anomaly detection) using tools like MLlib or scikit-learn without any ETL rewrite; (3) Separation of concerns—decoupling the analytical layer from the operational database protects OLTP performance during batch analytics runs.

The five materialized views serve the real-time dashboard (sub-500ms), while the Spark pipeline handles deeper historical analysis.

In terms of the 5 V's: Volume is addressed by the 146K-row test dataset and the million-row production projection; Velocity is managed through scheduled ETL refresh (`REFRESH MATERIALIZED VIEW CONCURRENTLY`) rather than real-time stream processing, appropriate for daily attendance data; Variety is handled by ingesting five different relational tables into a unified Spark DataFrame; Veracity is enforced through parameterized SQL queries and the automated data quality checks in `spark_etl.py`; Value is realized through the five KPI dashboards enabling STB management to identify understaffed sites, absenteeism trends, and agent workload imbalances.

2.4 Development Methodology: Comparative Analysis and Selection

2.4.1 Waterfall Model

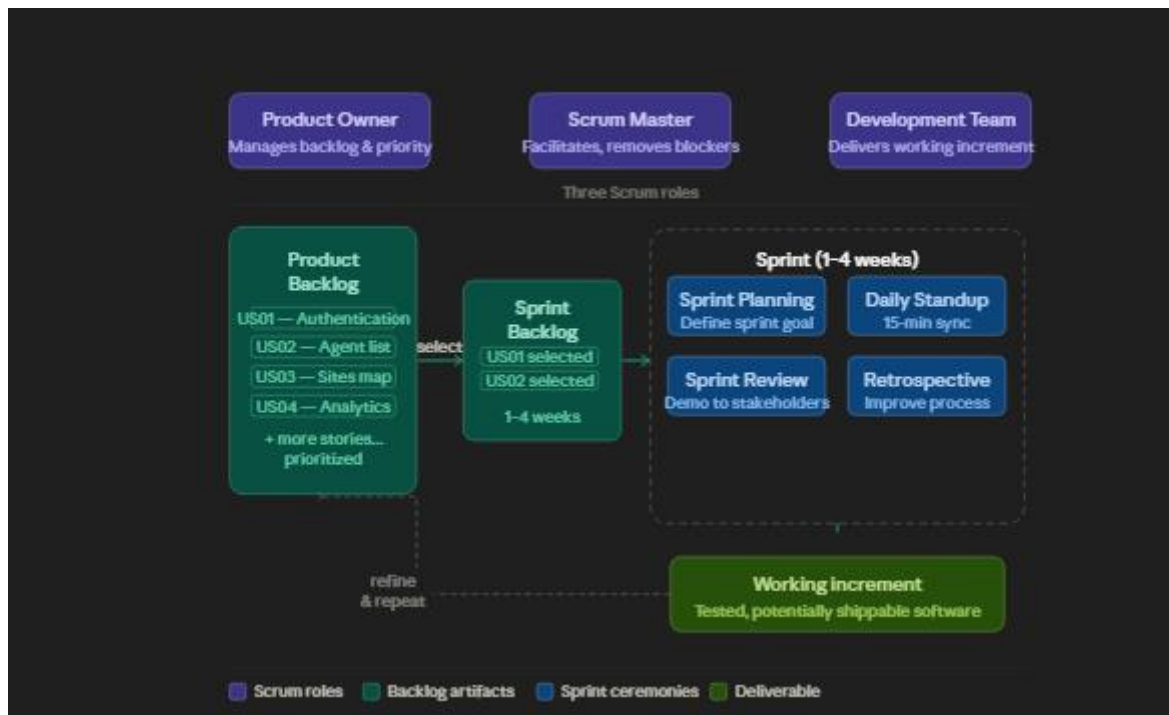
The Waterfall model (Royce, 1970) is a linear sequential approach: Requirements → Design → Implementation → Testing → Deployment. Each phase must complete before the next begins. Its strengths are clear structure, well-defined milestones, and comprehensive documentation. Its critical weaknesses are extreme inflexibility to changing requirements, late defect discovery (testing only after full implementation), and no working software until project ends. In modern web development where requirements evolve continuously, Waterfall is poorly suited.

2.4.2 Agile Methodology

Agile (Manifesto, 2001) is an iterative, incremental approach dividing work into short cycles (2–4 weeks), each producing a working software increment. Agile prioritizes individuals and interactions, working software, customer collaboration, and responding to change. Strengths: flexibility, early delivery, rapid feedback, reduced risk. Weaknesses: less predictable scope and timeline, requires strong communication, documentation may be insufficient.

2.4.3 Scrum Framework

Scrum is the most adopted Agile framework. It organizes work into fixed Sprints (1–4 weeks) with ceremonies: Sprint Planning, Daily Standup, Sprint Review, and Sprint Retrospective. Three roles: Product Owner (manages backlog), Scrum Master (facilitates process), Development Team (delivers increment). Work items are expressed as User Stories in a prioritized Product Backlog.



Scrum Sprint Cycle

2.4.4 Methodology Comparison

Criterion	Waterfall	Agile	Scrum
Flexibility to change	Very Low	Very High	Very High
Early delivery of value	No	Yes	Yes
Customer involvement	Low	Continuous	Continuous
Documentation	Extensive	Lightweight	Moderate
Risk management	Late detection	Early detection	Early detection
Team autonomy	Low	High	High
Suitable for evolving req.	No	Yes	Yes
Structure / ceremonies	Rigid	Minimal	Structured
Best suited for	Fixed-scope projects	Small agile teams	Structured agile teams

Table 2.1 – Methodology Comparison: Waterfall vs. Agile vs. Scrum

2.4.5 Selected Methodology: Agile-Inspired Iterative Approach

An Agile-inspired iterative and incremental approach was selected for this project. This choice is justified by four decisive factors:

- 1. Evolving requirements:** The full scope was not entirely defined at the outset. As discussions with STB stakeholders progressed, new requirements emerged (Big Data analytics, two-step approval workflow, avatar upload, contact support). An iterative approach absorbed these naturally — Waterfall would have required costly rework.
- 2. Solo development context:** Scrum's full ceremony set (Scrum Master, Product Owner, daily standups) is designed for teams. A lightweight Agile approach — short iterations, maintained product backlog, continuous functional delivery — was better adapted to a single-developer internship.
- 3. Early validation:** Delivering working features at the end of each iteration enabled early demonstrations to supervisors, gathering feedback before investing further effort in the wrong direction.
- 4. Risk reduction:** The most technically uncertain components — the Big Data module and the three-role RBAC system — were developed in early iterations, allowing their risks to be identified and mitigated before they could threaten the project timeline.

Development was organized into seven iterations: (1) Authentication and role infrastructure; (2) Agent, site, and assignment management; (3) Attendance tracking and incident reports; (4) Request/ticket workflow and contact support; (5) Big Data analytics module; (6) UI refinements and avatar upload; (7) Testing and documentation.

2.5 Technology Stack Justification

2.5.1 Backend Framework

Criterion	Node.js / Express 5	Django (Python)	Spring Boot (Java)
Language	JavaScript	Python	Java
Full-stack consistency	✓ (JS everywhere)	✗	✗
I/O Performance	Excellent	Good	Good
Learning curve	Low	Medium	High
npm ecosystem	Very rich	Rich	Rich
REST API design	Flexible	Opinionated	Opinionated
Selected	✓ Chosen	—	—

Table 2.2 – Backend Framework Comparison

Node.js/Express 5 was selected for its JavaScript/TypeScript consistency with the Angular frontend, excellent non-blocking I/O performance, flexible architecture, and mature npm ecosystem (jsonwebtoken, bcryptjs, multer, pg).

2.5.2 Frontend Framework

Criterion	Angular 21	React 18	Vue 3
Language	TypeScript (strict)	JSX / JavaScript	JavaScript / TS
Architecture	Full framework	Library	Progressive framework
Built-in routing	✓	✗ (React Router)	✗ (Vue Router)
Built-in HTTP client	✓ HttpClient	✗	✗
Interceptors	✓ Native	Manual	Manual
Reactive forms	✓ Native	External libs	VeeValidate
Selected	✓ Chosen	—	—

Table 2.3 – Frontend Framework Comparison

Angular 21 was selected for its structured standalone component architecture, built-in HttpClient with interceptor support for seamless JWT attachment, reactive forms for client-side validation, TypeScript's compile-time error catching, and its opinionated structure that enforces maintainable enterprise code.

2.5.3 Database

Criterion	PostgreSQL 16	MySQL 8	MongoDB
Type	Relational (SQL)	Relational (SQL)	Document (NoSQL)
ACID compliance	Full	Full	Partial
Foreign keys + CASCADE	✓	✓	✗
Materialized views	✓ Native	✗	✗
Window functions	✓ Full	Partial	✗
Suitable for relational data	✓ Excellent	✓ Good	✗
Selected	✓ Chosen	—	—

Table 2.4 – Database Comparison

PostgreSQL 16 was selected because the application data is inherently relational (7 entities with FK constraints). Its native materialized views were decisive for the Big Data analytics module, and its full ANSI SQL compliance with advanced window functions and ACID guarantees make it the optimal choice.

This chapter surveyed the key technologies, architectural patterns, and methodologies relevant to the STB Security project. Each technology was selected based on objective comparative criteria. The following chapter presents the requirements analysis, translating the problem into concrete specifications.

Chapter 3: Requirements Analysis

Requirements analysis defines precisely what the system must do, for whom, and under what constraints. This chapter identifies the system actors, specifies functional and non-functional requirements, presents use case diagrams, and establishes the product backlog.

3.1 System Actors

3.1.1 Administrator (Level 3 — Full Access)

The administrator is the system's principal manager with unrestricted access to all features: managing all agent profiles, bank sites, and assignments; managing user accounts, roles, and passwords; providing final approval or rejection of incident reports and agent requests; consulting Big Data analytics and KPI dashboards; and receiving support messages. This role is held by the STB IT manager or a designated management representative.

3.1.2 Team Leader / Chef d'Équipe (Level 2 — Intermediate)

The team leader supervises agents assigned to one or more bank sites. Capabilities: creating new agents (auto-assigned to the leader's sites for 90 days), recording daily attendance with time logs, managing team assignments, submitting incident reports, performing first-stage approval of agent requests, and managing team-level tickets. No global administration privileges.

3.1.3 Security Agent (Level 1 — Restricted)

The security agent has restricted personal access: logging in and managing own profile (name, email, phone, password, avatar), viewing current and past assignments, consulting monthly attendance history, submitting requests (leave, work certificate, support), and contacting the support team via a dedicated form.

3.2 Functional Requirements

Ref.	Feature	Description
FR01	Authentication	Login with email and password. JWT token generated and returned. Frontend stores token and attaches it to every HTTP request via interceptor.
FR02	Agent Management	Full CRUD on security agent profiles. New agents created by a team leader are auto-assigned to the leader's site for 90 days.
FR03	Site Management	Full CRUD on bank sites. One team leader per site enforced. Interactive Leaflet map for geographic visualization.
FR04	Assignment Management	Assign an agent to a site for a defined period. View and delete assignments. Filtered by site for team leaders.
FR05	Attendance Tracking	Record daily attendance (present, late, absent, on leave) with check-in/check-out times. Bulk recording. Monthly attendance grid.
FR06	Incident Reports	Team leaders create reports (incident, absence, health, other). Administrators validate or reject reports.
FR07	Request Workflow	Agents submit requests. Two-step approval: team leader (step 1) then administrator (step 2).
FR08	Dashboard	Real-time KPIs adapted to user role: active agents, sites, assignments, attendance, pending requests.
FR09	Access Control	Hierarchical RBAC: Agent(1), Team Leader(2), Admin(3). Backend middleware + Angular AuthGuard.
FR10	Profile Management	Any user can update personal info, change password (with current password verification), upload avatar.
FR11	User Management	Administrator manages all user accounts: view, change role, reset password.
FR12	Contact Support	Agents and team leaders send support messages. Administrator views all messages.
FR13	Big Data Analytics	Admin accesses: attendance trend, absenteeism by branch, incidents by type, agent workload, site coverage, KPI summary.
FR14	Logout	Removes JWT from localStorage, redirects to login.

Table 3.1 – Functional Requirements

3.3 Non-Functional Requirements

Category	Description
Security	JWT authentication, bcrypt password hashing (cost 10), CORS restriction, backend route guards, Angular AuthGuard.
Performance	API response < 500ms. Materialized views for complex analytics to avoid real-time aggregation overhead.
Availability	Accessible continuously during STB business hours.
Maintainability	Modular structure: standalone Angular components, separate Express route files per resource, environment configuration.
Usability	Intuitive role-adapted navigation, clear error messages, responsive layout.
Compatibility	Modern browsers: Chrome, Firefox, Edge.
Scalability	Analytics pipeline designed for Apache Spark cluster scaling.
Auditability	All modifications traceable via created_by and valide_par fields.

Table 3.2 – Non-Functional Requirements

3.4 Use Case Diagrams

The following diagram illustrates all interactions between the three system actors and the application's features. The system is divided into three sections: Agent use cases, Chef d'Équipe use cases, and Administrator use cases.

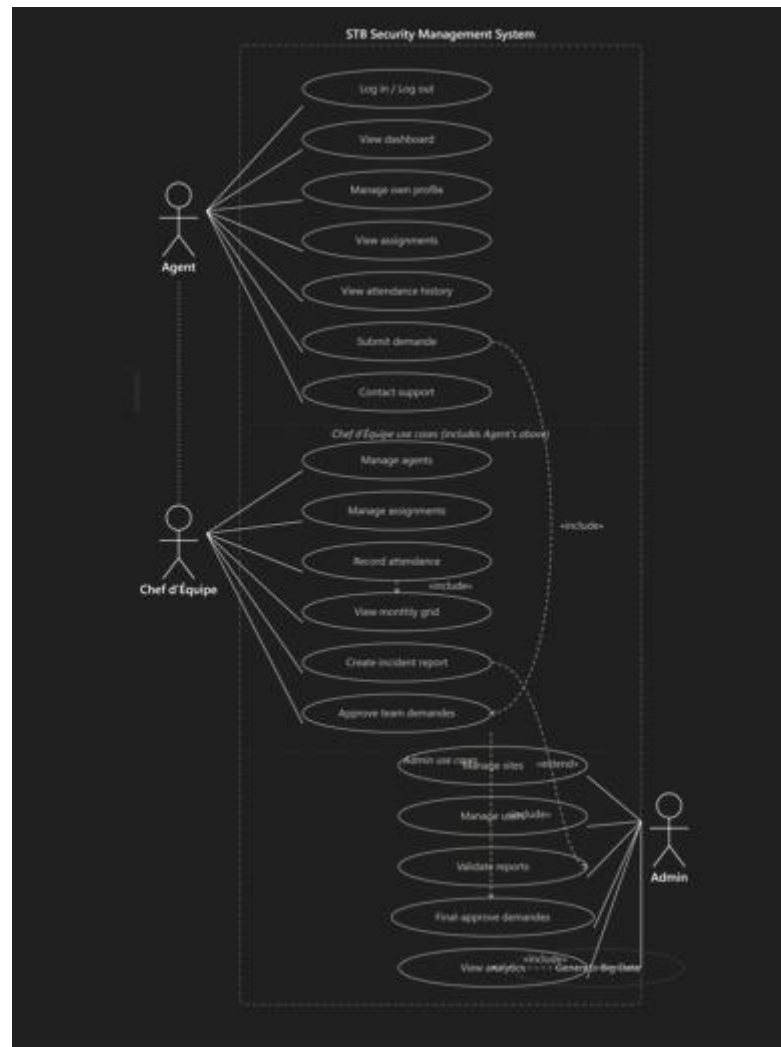


Figure 3.1 – Overall Use Case Diagram – STB Security Management System

The diagram confirms the hierarchical access model: the Agent actor accesses personal features only (Login, View assignments, View attendance, Submit request, View profile, Contact support). The Chef d'Équipe manages team operations (Manage agents, Create assignments, Record attendance, Manage team, Create reports, Approve requests). The Administrator holds full authority (Manage users, Manage sites, View analytics, Validate reports, Final approval, Manage system).

3.5 Product Backlog

The product backlog brings together all the project's user stories, prioritized and estimated in story points (SP). The estimate follows an adapted Fibonacci scale (1, 2, 3, 5, 8) reflecting the relative complexity and effort of each story.

ID	User Story	SP	Priorité	Statut
US01	Authentication (login, JWT)	5	High	Completed
US02	List of agents	3	High	Completed
US03	Adding an agent with auto-assignment	3	High	Completed
US04	Agent Modification	3	High	Completed
US05	Agent Removal (Cascading)	3	Medium	Completed
US06	Management of sites with map	5	High	Completed
US07	Agent-Site Assignments	5	High	Completed
US08	Dashboard by Role	5	High	Completed
US09	Hierarchical access control	5	High	Completed
US10	Attendance Clock	5	High	Completed
US11	Rapports d'incident	3	High	Completed
US12	Demandes à deux niveaux	5	High	Completed
US13	Profil et avatar	3	Medium	Completed
US14	Gestion des comptes	5	High	Completed
US15	Support	3	Medium	Completed
US16	Analytique et graphiques KPI	5	High	Completed
US17	Vue cartographique Leaflet	3	Medium	Completed
US18	Module Big Data (ETL PySpark)	5	High	Completed
US19	Vues matérialisées et prévision	5	High	Completed
US20	Recherche et filtrage	2	Bass	Postponed
US21	Pagination côté serveur	2	Bass	Postponed

Table 3.3 – Product Backlog (with story points)

Estimated total: 85 story points planned in the product backlog, including 81 SPs delivered (95%) and 4 SPs carried over to perspectives.

The priorities have been defined in collaboration with the STB's professional supervisor, according to the operational criticality and the order of the dependencies: authentication must precede all protected functionalities; the management of agents and sites must precede assignments; assignments must precede the clocking in.

3.6 Sprint breakdown

The development was organized in seven sprints covering the period from February 2 to May 29, 2026. Each sprint, lasting two to three weeks, delivers a testable functional increment and concludes with a review with the supervisors. The following figure shows the overall planning.

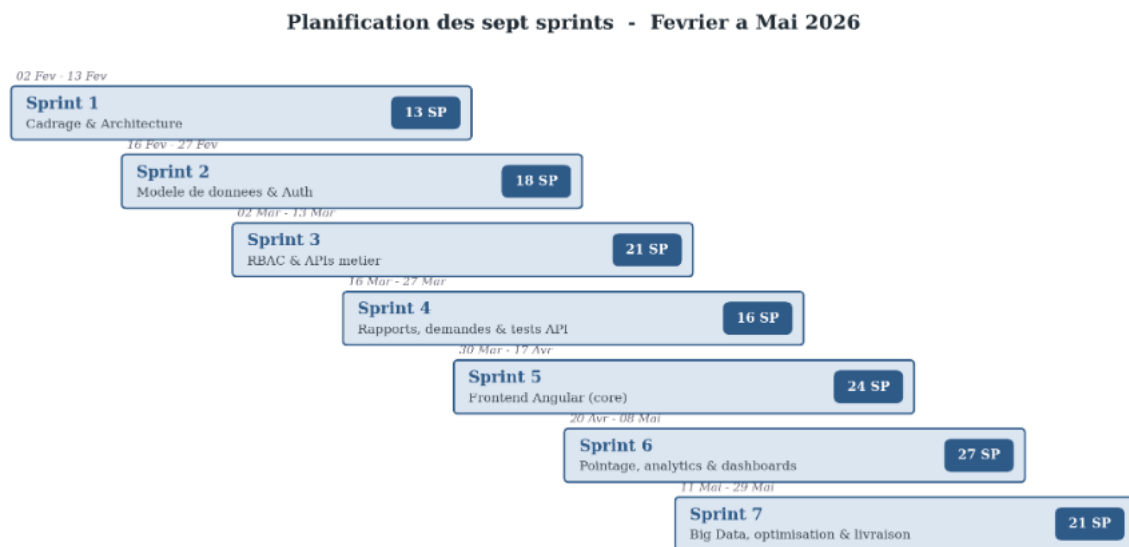


Figure 3.2 – Planning for the seven sprints (February – May 2026)

The overall burndown below tracks the consumption of story points sprint after sprint, compared to the ideal trajectory. At the sprint level, product backlog epics are broken down into more detailed user stories, bringing the total to 140 SPs. The proximity of the two curves testifies to a regular velocity and a good control of the perimeter.

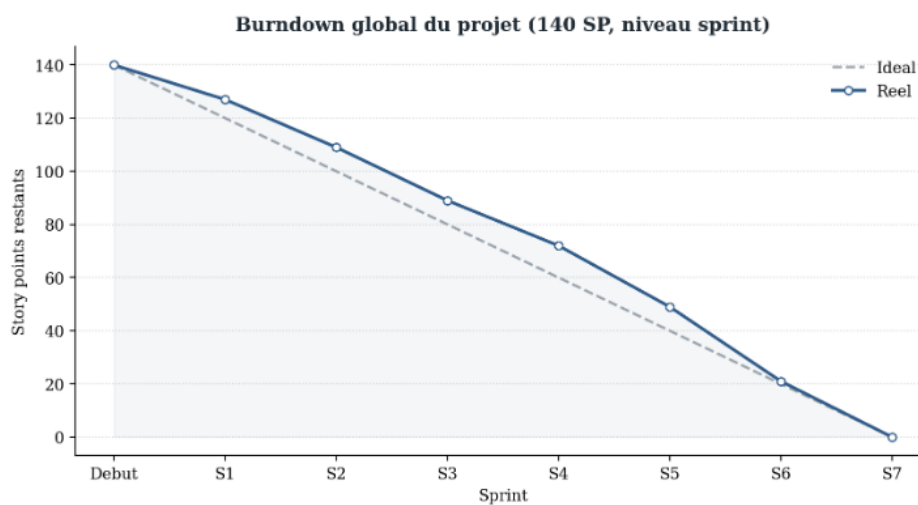


Figure 3.3 – Overall project burndown (140 SPs at the sprint level)

3.6.1 Sprint 1 — Scoping and Architecture

Duration: 2 weeks (02 February – 13 February 2026). Velocity engaged: 13 story points.

Objective of the sprint: Frame the project, gather the needs, choose the technology stack and design the overall architecture of the application.

ID	User Story	SP	Status
US01	Collection of needs and drafting of specifications	5	Completed
US02	Technology benchmark and stack choice	3	Completed
US03	Environment Setup (Node.js, PostgreSQL, Angular CLI)	3	Completed
US04	Three-thirds architecture design	2	Completed

Table 3.4 – Sprint Backlog of Sprint 1

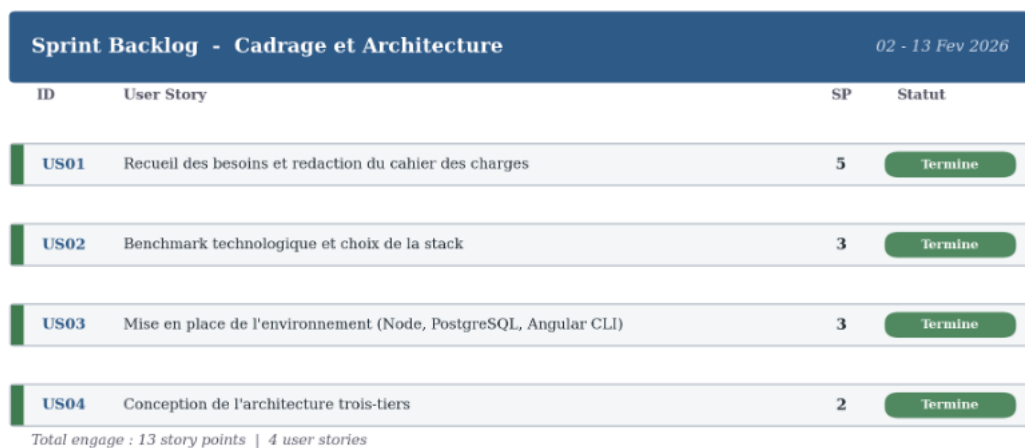


Figure 3.4 – Sprint Backlog visuel – Sprint 1

Sprint review: The functional scope and the technology stack (Angular, Node.js/Express, PostgreSQL, Apache Spark) have been validated with the professional coach. The development environment is up and running and the Git repository is initialized.

3.6.2 Sprint 2 — Data Model and Authentication

Duration: 2 weeks (February 16 – February 27, 2026). Velocity engaged: 18 story points.

Objective of the sprint: Model the relational database and set up the secure authentication layer (JWT, bcrypt).

ID	User Story	SP	Status
US05	Database Modeling (ERD, 7 entities)	5	Completed
US06	SQL scripting: creating tables, CHECK constraints, and indexes	3	Completed
US07	Express Server, CORS Configuration, and Basic Routing	2	Completed
US08	Registration/login with bcrypt hash and JWT generation	5	Completed
US09	JWT token verification middleware	3	Completed

Table 3.5 – Sprint Backlog of Sprint 2

Sprint Backlog - Modele de donnees et Authentification			16 - 27 Feb 2026
ID	User Story	SP	Statut
US05	Modelisation de la base (ERD, 7 entites)	5	Termine
US06	Script SQL : tables, contraintes, index	3	Termine
US07	Serveur Express, CORS et routage de base	2	Termine
US08	Inscription / connexion avec bcrypt et JWT	5	Termine
US09	Middleware de verification du jeton JWT	3	Termine

Total engage : 18 story points | 5 user stories

Figure 3.5 – Sprint Backlog visuel – Sprint 2

Sprint review: The seven-entity relationship schema is in place with its integrity constraints. Stateless authentication by JWT token works and has been tested with Postman; Protected routes correctly return a 401 error when there is no valid token.

3.6.3 Sprint 3 — RBAC et APIs métier

Durée : 2 semaines (02 mars – 13 mars 2026). **Vélocité engagée :** 21 story points.

Objectif du sprint : Implémenter le contrôle d'accès hiérarchique à trois niveaux et développer les APIs métier principales (agents, sites, affectations, présences).

ID	User Story	SP	Statut
US10	Middleware RBAC hiérarchique (Agent 1, Chef 2, Admin 3)	5	Terminé
US11	API agents (CRUD, jointure avec la table users)	3	Terminé
US12	API sites (règle un chef par site, transactions)	3	Terminé
US13	API affectations (plages de dates, anti-chevauchement)	5	Terminé
US14	API présences (pointage, calcul du statut, grille mensuelle)	5	Terminé

Tableau 3.6 – Sprint Backlog du Sprint 3

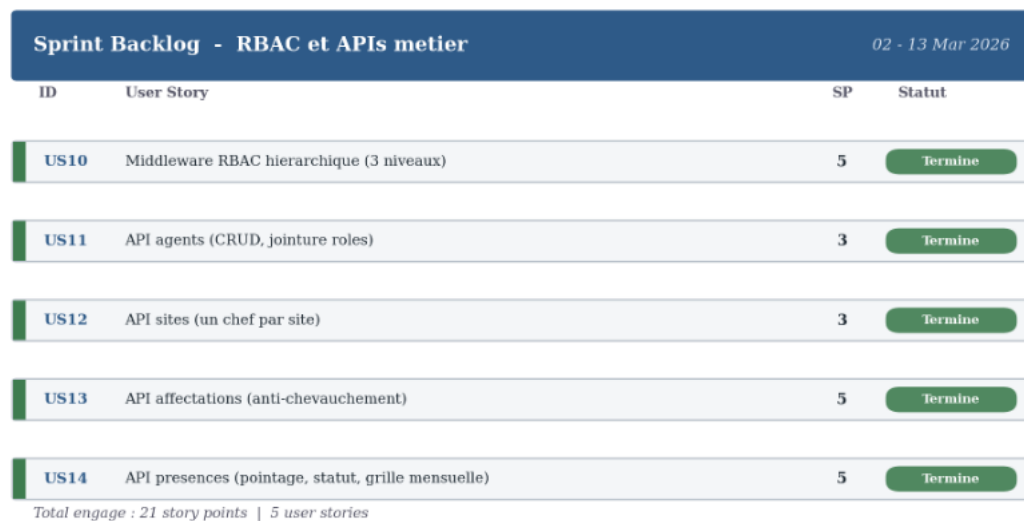


Figure 3.6 – Sprint Backlog visuel – Sprint 3

Revue de sprint : Le cœur métier de l'API est fonctionnel. Le RBAC distingue correctement les rôles, l'affectation interdit le chevauchement via l'opérateur de plage de dates de PostgreSQL, et le pointage calcule automatiquement le statut (retard / à l'heure) par rapport à l'horaire théorique.

3.6.4 Sprint 4 — Rapports, demandes et tests API

Durée : 2 semaines (16 mars – 27 mars 2026). **Vélocité engagée :** 16 story points.

Objectif du sprint : Compléter les fonctionnalités backend (rapports d'incident, workflow de demandes, support, upload) et fiabiliser l'API par des tests.

ID	User Story	SP	Statut
US15	Crash Reporting API + admin validation endpoint	3	Completed
US16	Requests API: Two-level approval workflow	5	Completed
US17	API support and avatar upload with Multer	3	Completed
US18	Postman testing (~50 queries) and bug fixes	5	Completed

Table 3.7 – Sprint Backlog of Sprint 4

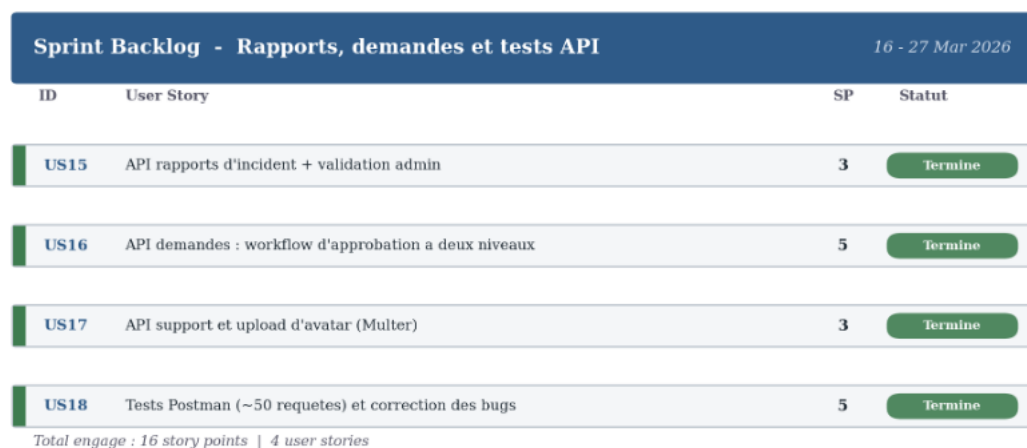


Figure 3.7 – Sprint Backlog visuel – Sprint 4

Sprint Review : The backend is comprehensive and robust. The two-level approval workflow (leader and then administrator) is up and running, avatars are validated and stored securely, and the Postman collection has made it possible to detect and correct several anomalies (500 errors on inconsistent dates, asynchronous error handling).

3.6.5 Sprint 5 — Frontend Angular (core)

Duration : 3 weeks (March 30 – April 17, 2026). Velocity engaged: 24 story points.

Objective of the sprint : Build the Angular interface: client-side authentication, role-based dashboards, agent, site and assignment management.

ID	User Story	SP	Statut
US19	Authentication Service, AuthGuard and JWT Interceptor	5	Completed
US20	Login page (responsive form)	3	Completed
US21	Dashboards tailored to all three roles	5	Completed
US22	Agent management (list, form, avatar upload)	5	Completed
US23	Site management with Leaflet interactive map	3	Completed
US24	Assignments module (list, creation, closure)	3	Completed

Table 3.8 – Sprint Backlog of Sprint 5

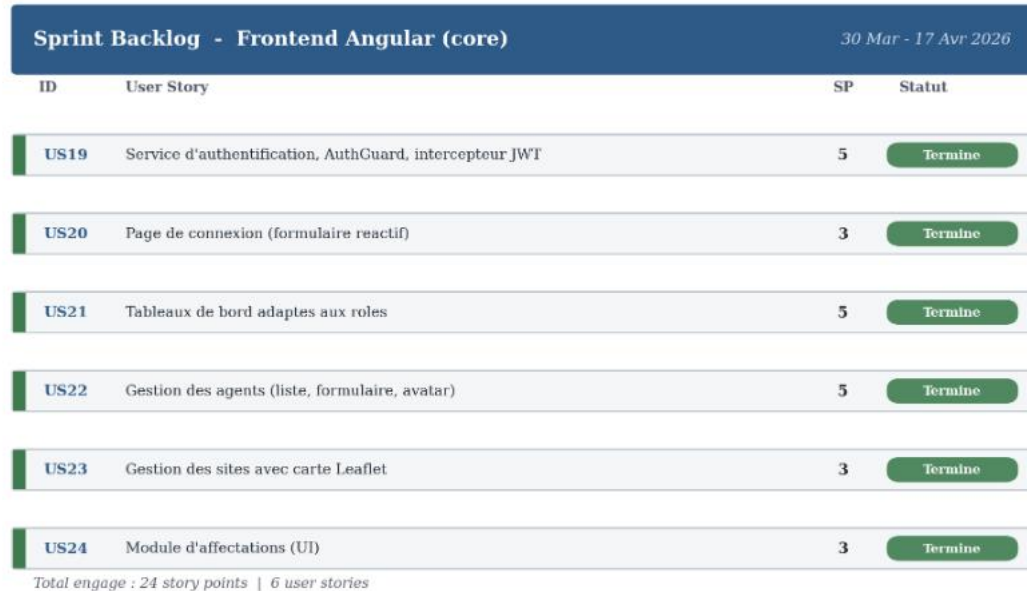


Figure 3.8 – Sprint Backlog visuel – Sprint 5

Sprint Review : The customer part is taking shape. The man-in-the-middle automatically attaches the JWT token to each request, the AuthGuard redirects unauthorized access, and the dashboards display indicators appropriate to the connected role. The Leaflet map visualizes banking sites with their real details.

3.6.6 Sprint 6 — Pointage, analytics et tableaux de bord

Duration : 3 weeks (20 April – 08 May 2026). Velocity engaged: 27 story points.

Objective of the sprint : Finalize operational screens (clocking, reporting, support) and deliver the analytics module based on materialized views.

ID	User Story	SP	Statut
US25	Punching in and down (head view, bulk check-in)	5	Completed
US26	Monthly attendance grid (×-day agent matrix)	5	Completed
US27	Incident reports and support module screens	3	Completed
US28	PostgreSQL Materialized Views for Analytics	5	Completed
US29	Analytics API + Forecasting Endpoint (Linear Regression)	5	Completed
US30	Big Data Dashboard with Charts (Chart.js)	4	Completed

Table 3.9 – Sprint Backlog of Sprint 6

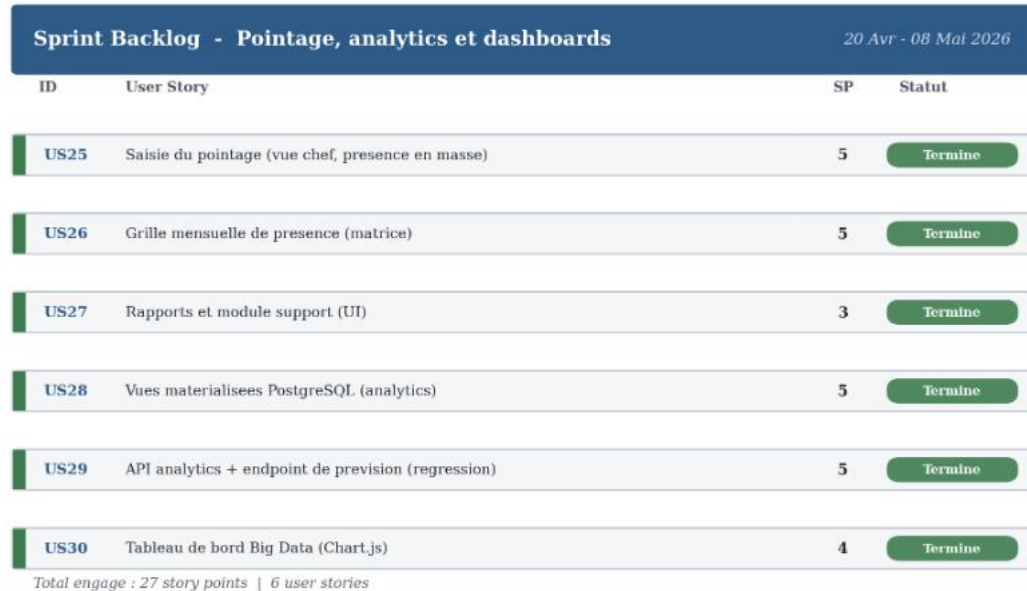


Figure 3.9 – Sprint Backlog visuel – Sprint 6

Sprint Review : The analytics module is delivered. The five materialized views feed into a real-time dashboard (under 500 ms), and the forecasting endpoint calculates the absenteeism trend by branch using PostgreSQL regression functions. The graphs make trends readable for management.

3.6.7 Sprint 7 — Big Data, optimisation et livraison

Duration : 3 weeks (May 11 – May 29, 2026). Committed velocity: 21 story points.

Objective of the sprint : Demonstrate scalability via the Spark prototype, optimize performance, and prepare the final deliverable.

ID	User Story	SP	Status
US31	ETL PySpark (lecture JDBC, DataFrames, sortie Parquet)	5	Completed
US32	Large-scale data generation (~146,000 rows)	3	Completed
US33	Scheduled refresh of views (node-cron)	3	Completed
US34	Performance Optimization (Composite Indexes)	3	Completed
US35	Documentation (ARCHITECTURE.md, README) and final deliverable	3	Completed
US36	Advanced search and filtering	2	Postponed
US37	Server-side pagination	2	Postponed

Table 3.10 – Sprint Backlog of Sprint 7

Sprint Backlog - Big Data, optimisation et livraison			11 - 29 Mai 2026
ID	User Story	SP	Statut
US31	ETL PySpark (JDBC, DataFrames, Parquet)	5	Termine
US32	Generation de donnees a grande echelle (~146k)	3	Termine
US33	Rafraichissement planifie des vues (node-cron)	3	Termine
US34	Optimisation des performances (index composites)	3	Termine
US35	Documentation et livrable final	3	Termine
US36	Recherche et filtrage avances	2	A faire
US37	Pagination cote serveur	2	A faire

Total engage : 21 story points | 7 user stories

Figure 3.10 – Sprint Backlog visuel – Sprint 7

Sprint Review : The Spark prototype processes the test dataset (~146,000 attendance) in less than a minute and writes the results to Parquet, demonstrating the scalability. The composite indices provide a gain of around $10\times$ on the monthly scale. Two user stories (search/filtering and server pagination) are reported in perspective.

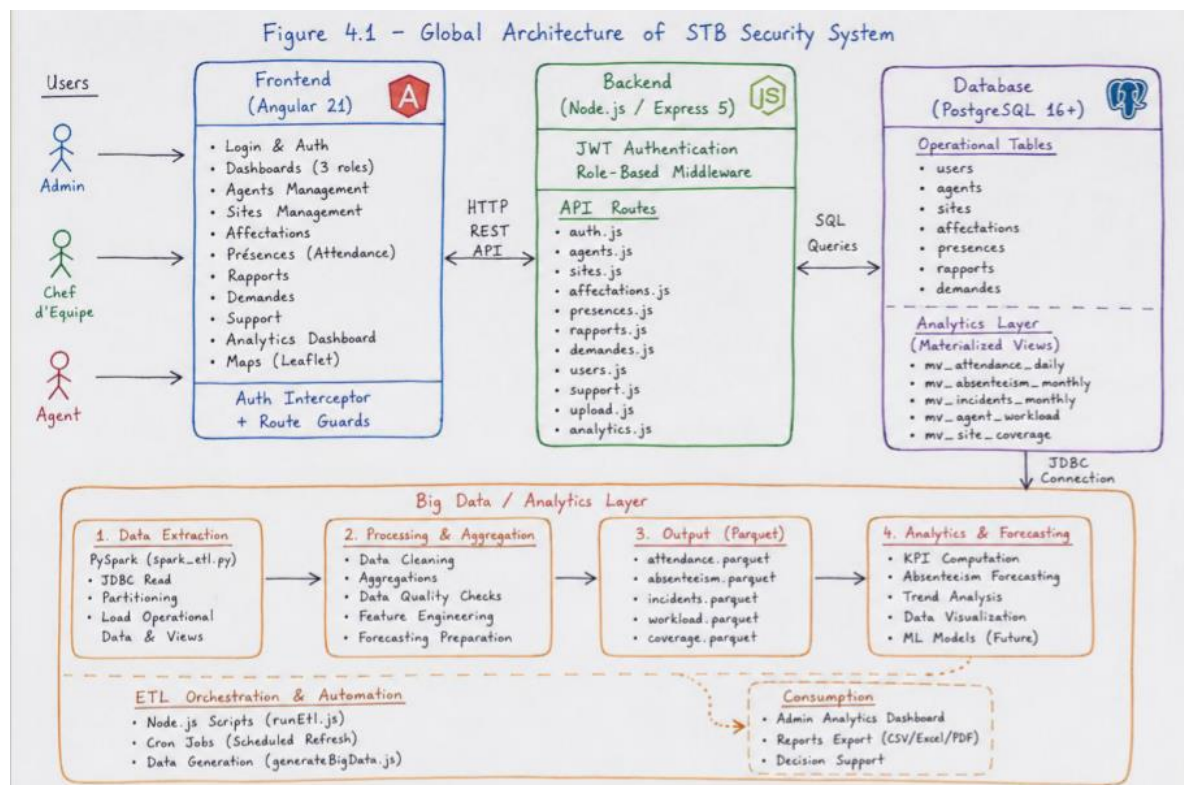
Chapter 4: System Design

The design phase translates requirements into a coherent technical architecture. This chapter presents the overall system architecture, the data model with its seven entities, the role permissions matrix, and the sequence diagrams for key workflows.

4.1 Overall System Architecture

The STB Security application follows a three-tier architecture separating presentation, business logic, and data persistence:

- Frontend (Angular 21 SPA): communicates via HTTP REST. JWT interceptor attaches token to every request. AuthGuard protects routes by role level. Runs on port 4200.
- Backend (Node.js / Express 5): RESTful API with 11 route modules. Middleware chains enforce access control (isAuth → isChefOrAdmin → isAdmin). Runs on port 3000.
- Database (PostgreSQL 16): 7 entities with enforced referential integrity, CASCADE constraints, and 6 materialized views for the analytics module.



System Architecture (Full 3-Tier with component details)

4.2 Data Model

4.2.1 Entity Descriptions

Entity	Description
users	User accounts: id, nom, email, password (bcrypt), role (agent/chef_equipe/admin), agent_id (FK→agents), avatar_url.
agents	Security agent profiles: id, nom, prenom, telephone, adresse, statut (actif/inactif), chef_id (FK→users).
sites	Bank sites: id, nom, adresse, ville, statut (actif/inactif), chef_id (FK→users, one per site).
affectations	Agent-site assignments (many-to-many junction): id, agent_id, site_id, date_debut, date_fin, statut, chef_id.
presences	Daily attendance: id, agent_id, site_id, date, heure_arrivee, heure_depart, statut (present/absent/retard/conge).
rapports	Incident reports: id, agent_id, site_id, type, contenu, date, statut, created_by (chef FK), valide_par (admin FK), created_at.
demandes	Requests: id, agent_id, type, date_debut, date_fin, motif, statut, chef_approved (boolean, step 1), valide_par (admin FK, step 2), created_at.

Table 4.1 – Database Entities Description

4.2.2 Entity-Relationship Diagram

The following ERD shows all seven entities and their relationships. The affectations table implements the many-to-many relationship between agents and sites. The demandes table shows the two-step approval structure through chef_approved (boolean) and valide_par (FK to admin). All foreign keys include ON DELETE CASCADE to maintain referential integrity.

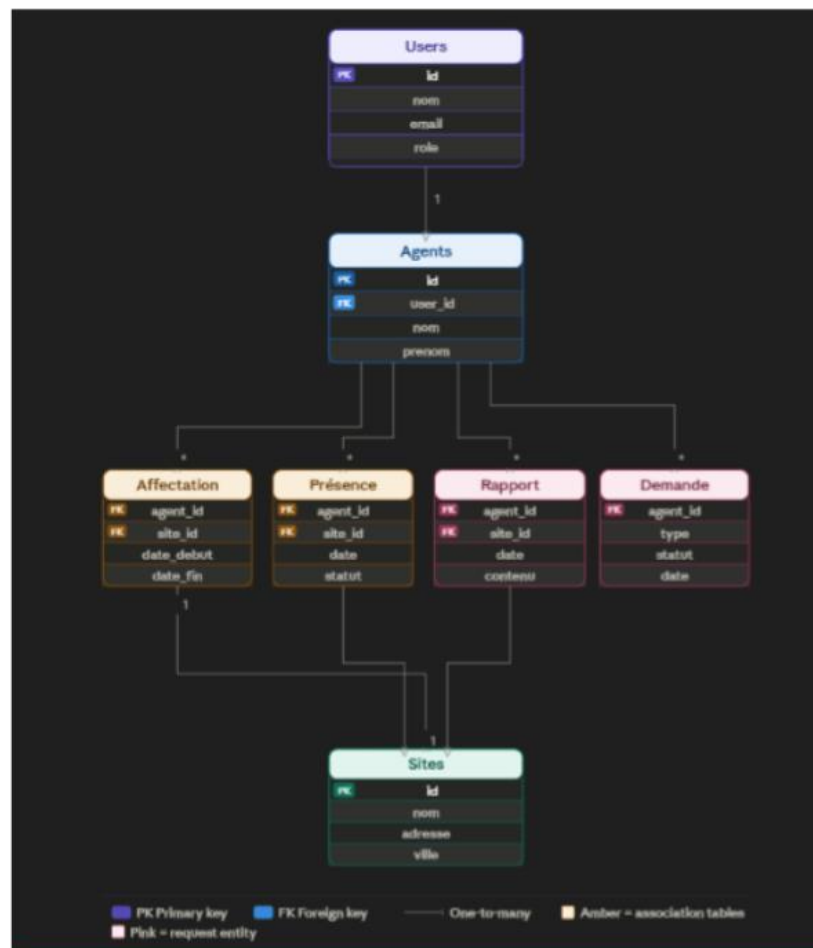


Figure 4.2 – Entity-Relationship Diagram (7 Entities)

4.2.3 Key Relationships

The affectations table is the many-to-many junction between agents and sites: one agent can be assigned to multiple sites over time, and one site can host multiple agents. ON DELETE CASCADE propagates deletions automatically. The demandes two-step approval uses chef_approved (boolean, set by team leader) and valide_par (FK to users.id, set by administrator). The rapports table references users twice: created_by (team leader) and valide_par (administrator), enabling full auditability.

4.3 Role Permissions Matrix

Feature	Agent	Team Leader	Admin
Dashboard (role-adapted KPIs)	✓	✓	✓
View own assignments	✓	✓	✓
Profile management & avatar	✓	✓	✓
Contact support (submit)	✓	✓	✗
Submit requests	✓	✗	✗
View own attendance history	✓	✓	✓
Team management	✗	✓	✓
Record attendance (pointage)	✗	✓	✓
Create incident reports	✗	✓	✗
Request approval (step 1)	✗	✓	✗
Agent management	✗	✓	✓
Site management & map	✗	✗	✓
User account management	✗	✗	✓
Validate incident reports	✗	✗	✓
Request final approval (step 2)	✗	✗	✓
Big Data analytics	✗	✗	✓
View support messages	✗	✗	✓

Table 4.2 – Role Permissions Matrix

4.4 Sequence Diagrams

4.4.1 JWT Authentication

The following diagram illustrates the nine-step login flow: (1) User submits credentials via Angular login form. (2) Angular sends POST /api/auth/login to Express API. (3) Express queries PostgreSQL for the user record. (4) PostgreSQL returns the user data. (5) Express calls bcrypt.verify(password, hash). (6) Express calls jwt.sign(payload) with the secret key. (7) Express returns 200 OK with the token. (8) Angular calls localStorage.setItem() to store the token. (9) Angular redirects to the role-appropriate dashboard.

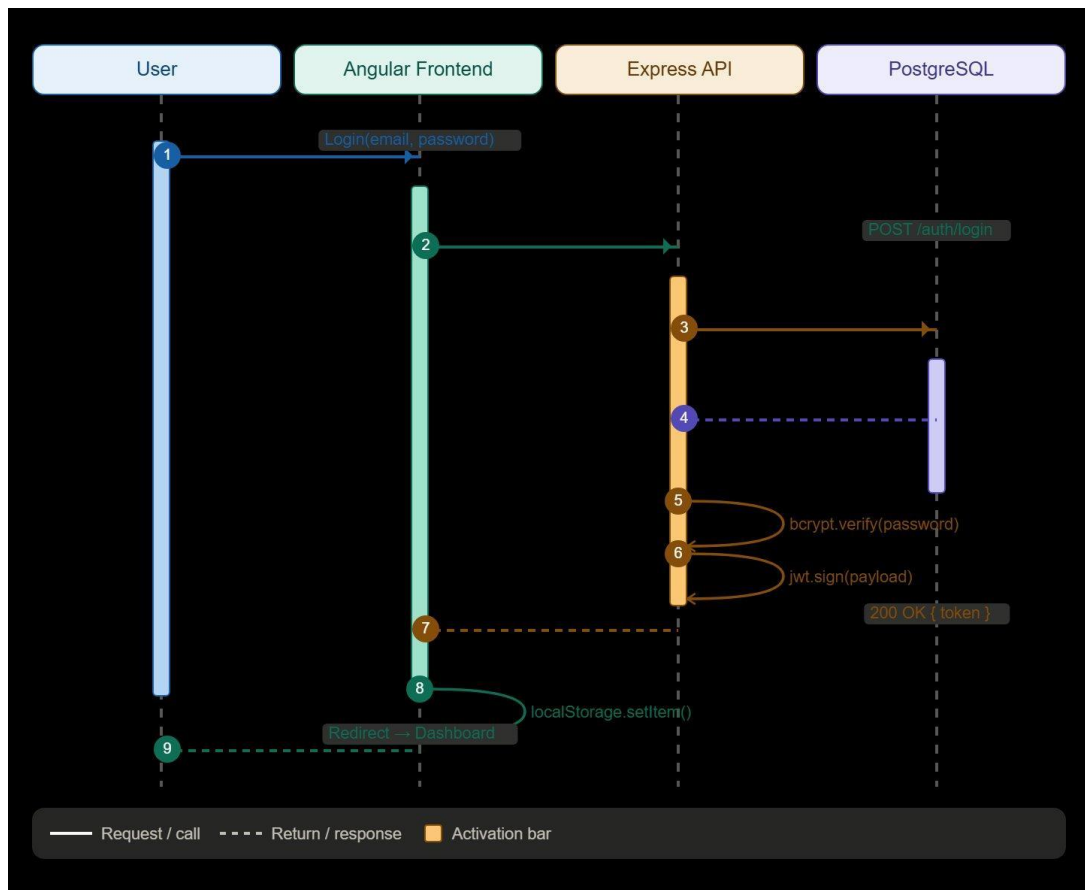


Figure 4.3 – Sequence Diagram – JWT Authentication

4.4.2 Agent Creation with Auto-Assignment

The team leader submits the agent creation form. Angular sends POST /api/agents. The isAuth middleware validates the JWT; isChefOrAdmin verifies role level ≥ 2 . The controller inserts the new agent into the agents table, then automatically creates an affectations entry linking the agent to the team leader's site with date_debut = today and date_fin = today + 90 days. The created agent and assignment are returned to the frontend.

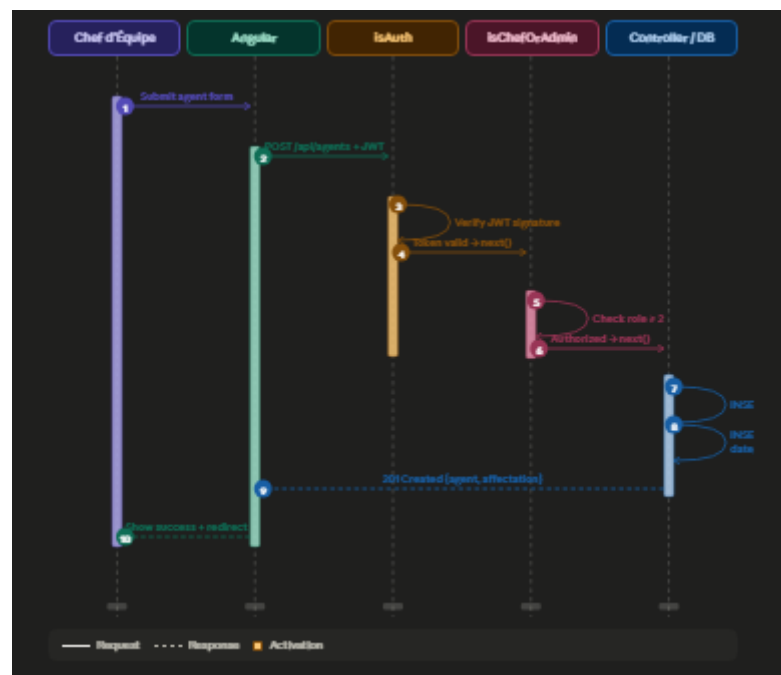


Figure 4.4 – Sequence Diagram – Agent Creation with Auto-Assignment

4.4.3 Two-Step Request Approval

Step 1 — Agent submits: POST /api/demandes/my-requests creates a demande with statut='pending' and chef_approved=false. Step 2 — Team leader approves: PUT /api/demandes/:id/chef-approve sets chef_approved=true. Step 3 — Administrator approves: PUT /api/demandes/:id/admin-approve sets statut='approved' and valide_par=admin.id. Any actor can also reject at their step, setting statut='rejected'.

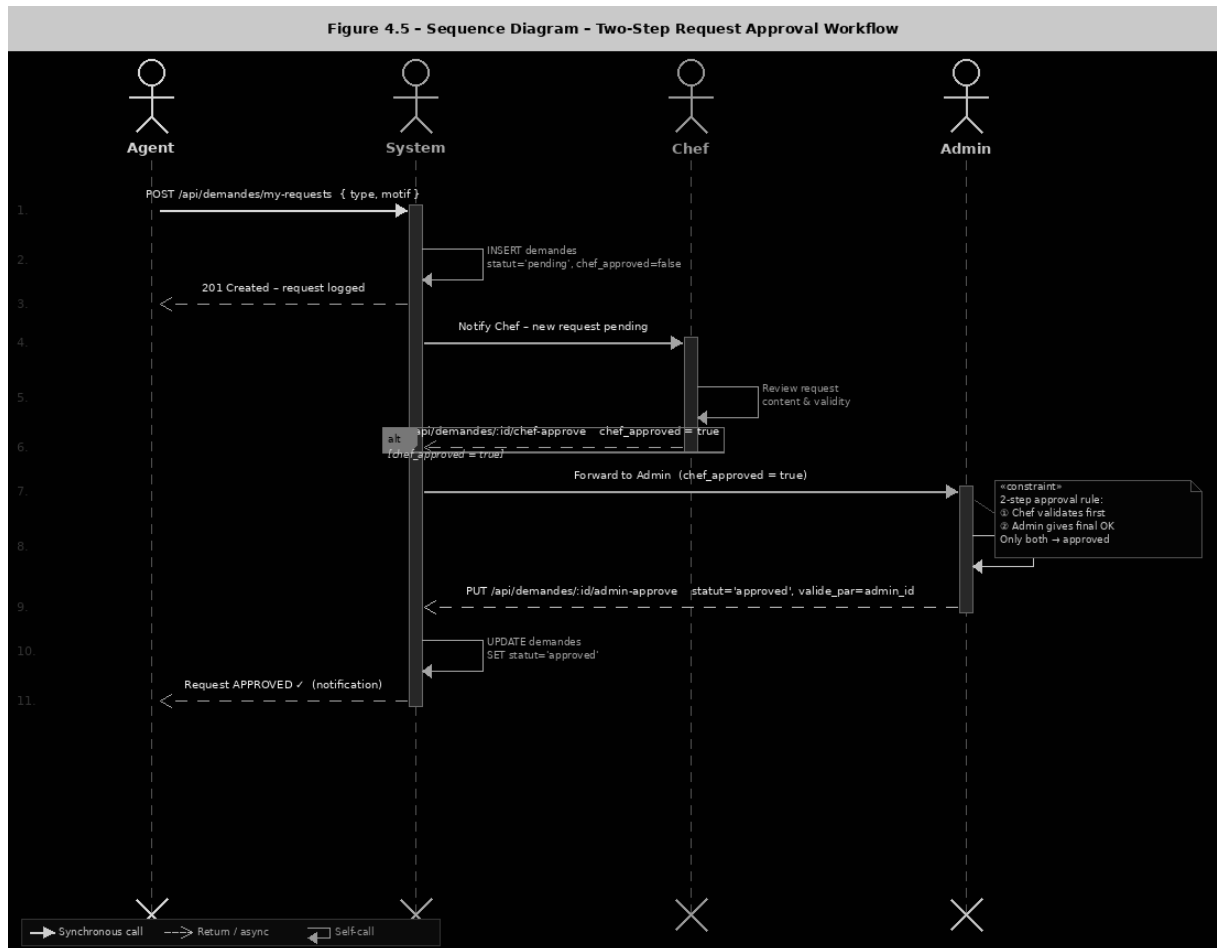


Figure 4.5 – Sequence Diagram – Two-Step Request Approval Workflow

This chapter established the technical architecture: three-tier structure, seven-entity data model with ERD, full permissions matrix, and the sequence flows of key processes. The following chapter presents the concrete implementation.

Chapter 5: Implementation and Testing

This chapter presents the concrete realization of the project. We describe the development environment, backend and frontend implementation, the security layer, the Big Data analytics module, and the testing results.

5.1 Development Environment

Component	Version / Detail
Operating System	Windows 10
Code Editor	Visual Studio Code 1.88
Node.js	v20.x (LTS)
npm	v10.x
Angular CLI	v19.x
PostgreSQL	v16
pgAdmin	v8
Postman	v11 (API testing)
Browser	Google Chrome
Version Control	Git
Python (Big Data)	3.11.x
Apache Spark	3.5.x (local prototype)

Table 5.1 – Development Environment

5.2 Backend Implementation

5.2.1 Project Structure

The backend follows a modular structure:

- `config/db.js`: PostgreSQL connection pool via `pg` module and `dotenv` environment variables.
- `middleware/auth.js` (`isAuth`): extracts Bearer token from Authorization header, verifies signature with `JWT_SECRET`, attaches decoded user to `req.user`.
- `middleware/roles.js`: `isChefOrAdmin` (`level ≥ 2`) and `isAdmin` (`level = 3`) middleware chaining after `isAuth`.
- `routes/`: 11 separate route files — `auth`, `agents`, `sites`, `affectations`, `presences`, `rappports`, `demandes`, `users`, `support`, `upload`, `analytics`, `dashboard`.
- `scripts/`: `seed.js`, `seedSites.js`, `generateBigData.js`, `runEtl.js`, `analytics_schema.sql`.

5.2.2 REST API Endpoints

Method	Endpoint	Role	Description
POST	/api/auth/login	Public	Authenticate, generate JWT
GET	/api/auth/me	Any	Get current user profile
GET	/api/agents	Chief+	List agents (site-filtered for chiefs)
POST	/api/agents	Chief+	Create agent + auto-assign 90 days
PUT	/api/agents/:id	Admin	Update agent profile
DELETE	/api/agents/:id	Admin	Delete agent (cascade)
GET	/api/sites	Any	List all bank sites
POST	/api/sites	Admin	Create site (one chief per site)
PUT	/api/sites/:id	Admin	Update site
DELETE	/api/sites/:id	Admin	Delete site
GET	/api/affectations	Chief+	List assignments with JOINS
POST	/api/affectations	Chief+	Create assignment
POST	/api/presences/bulk	Chief+	Bulk record attendance
GET	/api/presences/monthly/:month	Chief+	Monthly attendance grid
GET	/api/presences/me/monthly/:month	Agent	Own monthly attendance
POST	/api/rapports	Chief+	Create incident report
PUT	/api/rapports/:id/validate	Admin	Validate/reject report
POST	/api/demandes/my-requests	Agent	Submit a request
PUT	/api/demandes/:id/chef-approve	Chief	Step 1: chief approval
PUT	/api/demandes/:id/admin-approve	Admin	Step 2: admin final approval
POST	/api/support	Agent+	Send support message
GET	/api/support	Admin	View support messages
POST	/api/upload/avatar	Any	Upload profile picture (multer)
GET	/api/analytics/summary	Admin	Global KPI summary
GET	/api/analytics/attendance-trend	Admin	Monthly attendance trend
GET	/api/analytics/absenteeism-by-branch	Admin	Absenteeism by branch
GET	/api/analytics/incidents-monthly	Admin	Monthly incidents by type

Method	Endpoint	Role	Description
GET	/api/analytics/agent-workload	Admin	Agent workload stats
GET	/api/analytics/coverage	Admin	Site coverage metrics
GET	/api/dashboard/stats	Any	Role-adapted dashboard counters

Table 5.2 – REST API Endpoints Summary

5.3 Frontend Implementation

5.3.1 Angular 21 Standalone Architecture

The project uses Angular 21 with standalone components (no NgModules). Each component declares its own imports, making the dependency graph explicit and enabling tree-shaking for smaller bundle sizes. The application includes 18 page components organized by role, 5 services, 1 AuthGuard, and 1 JWT interceptor.

The AuthGuard reads the user's role from localStorage and compares it against the route's data.allowedRoles minimum level using a role-level map (agent=1, chef_equipe=2, admin=3). Insufficient role triggers a redirect to the dashboard rather than exposing a 403 page.

5.3.2 Application Screens

The following screenshots illustrate the main application interfaces:

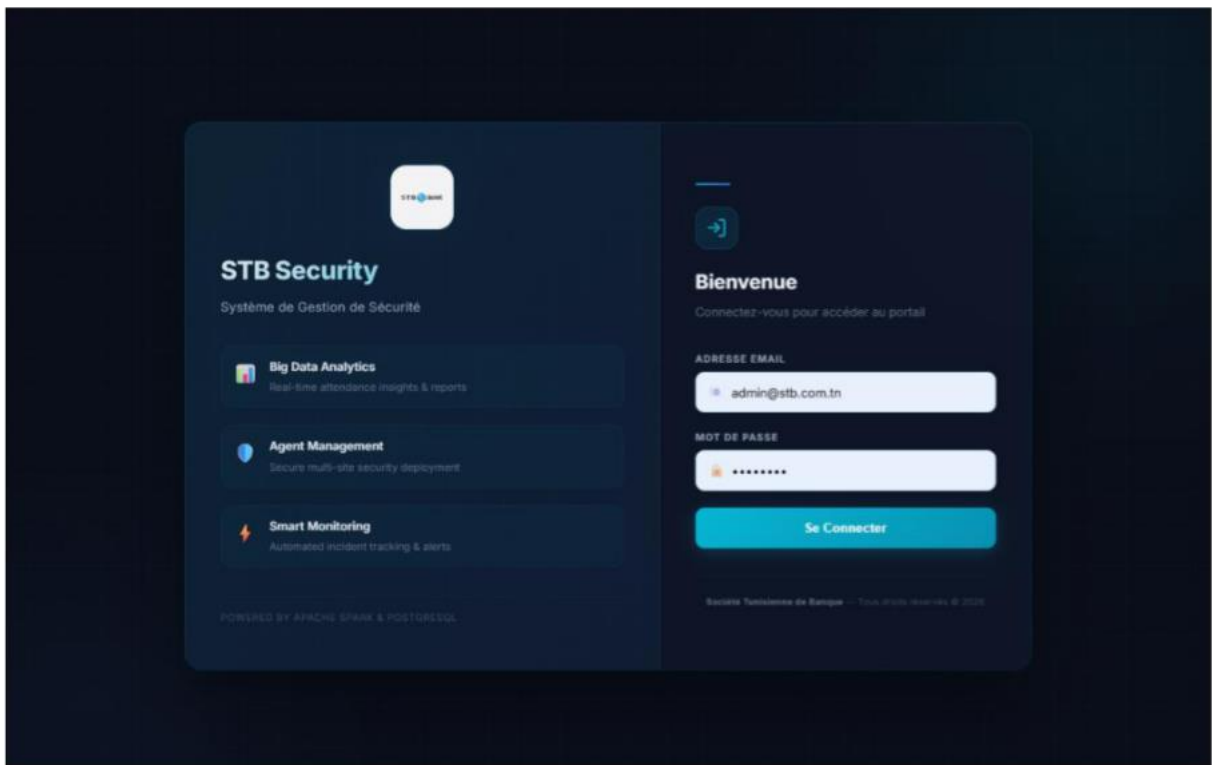


Figure 5.1 – Login Page

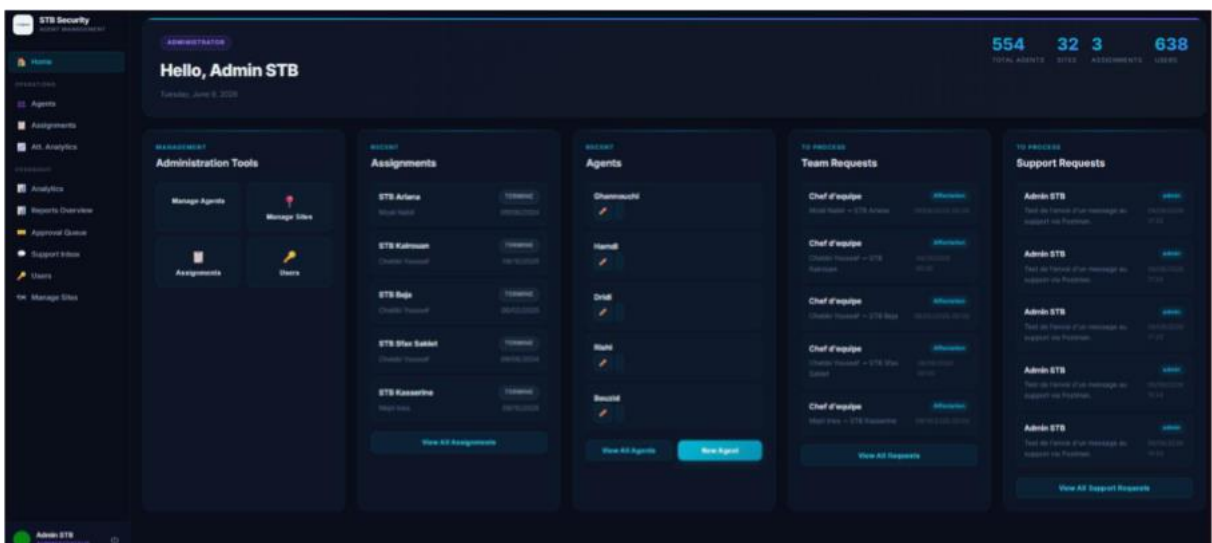


Figure 5.2 – Administrator Dashboard

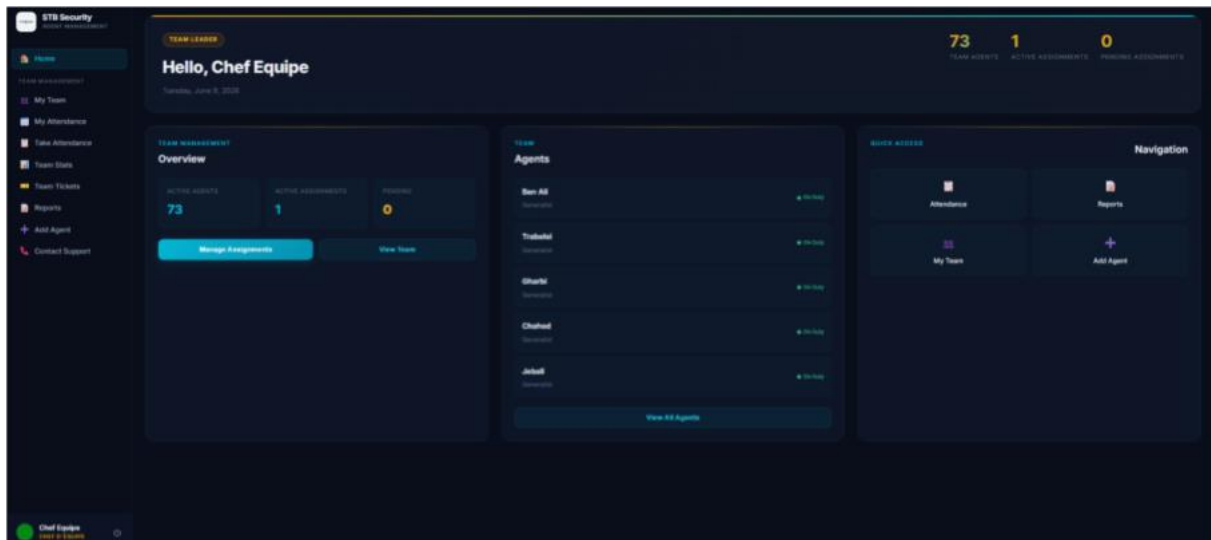


Figure 5.3 – Team Leader Dashboard

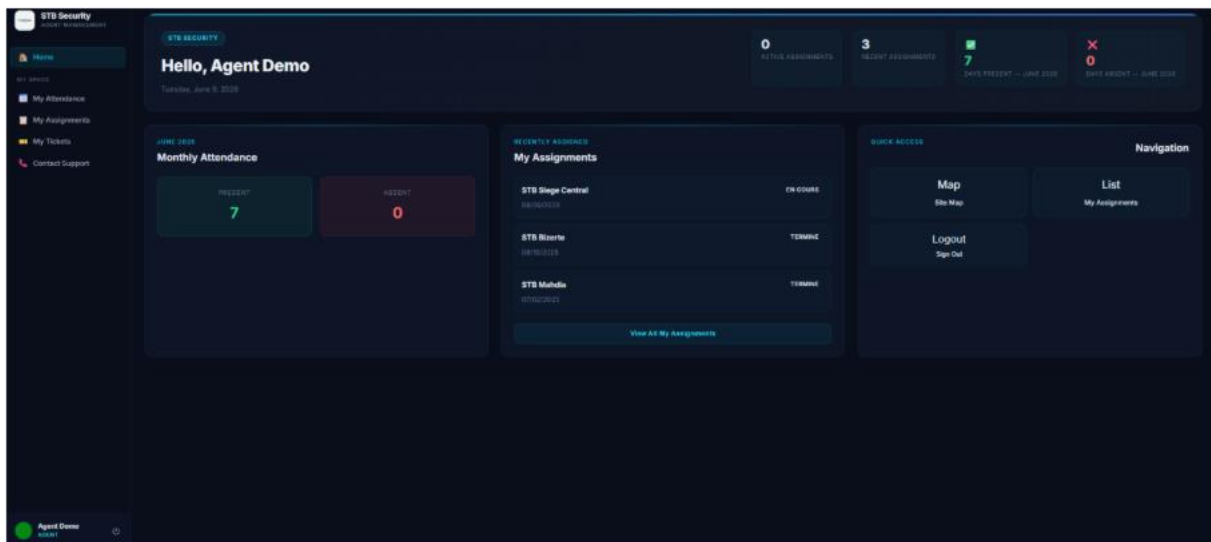


Figure 5.4 – Security Agent Dashboard

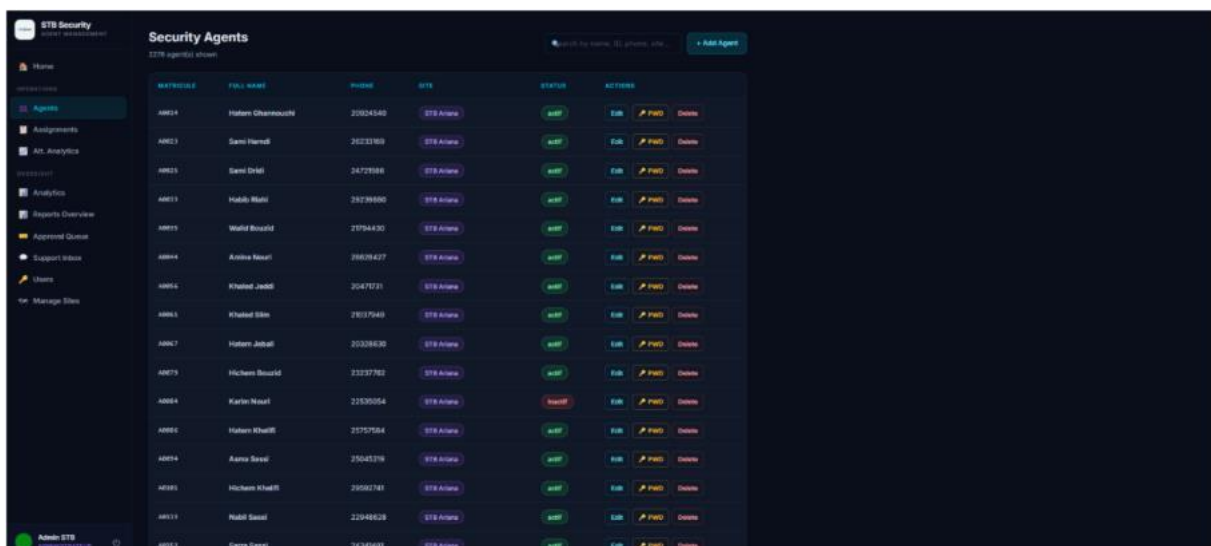


Figure 5.5 – Agent List Page

Add Agent
Register a new security agent

LAST NAME *
Ex: Ben Ali

FIRST NAME *
Ex: Karim

ID *
Ex: 100004

PHONE
Ex: 0023456 78 9010

ACCOUNT TYPE *
Regular Agent

STATUS
Active

Add Agent Cancel

Figure 5.6 – Add Agent Form

Manage Sites
22 sites

Name	City	Status	Actions
STB Stage Central	Tunis	active	edit delete
STB Tunis Lafayette	Tunis	active	edit delete
STB Tunis Bab Bhar	Tunis	active	edit delete
STB Tunis El Manakh	Tunis	active	edit delete
STB Tunis La Mersa	Tunis	active	edit delete
STB Ariana	Ariana	active	edit delete
STB Ben Ammi	Ben Ammi	active	edit delete
STB Monrovia	Monrovia	active	edit delete
STB Haddad	Haddad	active	edit delete
STB Hammam	Hammam	active	edit delete
STB Blouza	Blouza	active	edit delete
STB Zaghwan	Zaghwan	active	edit delete
STB Bepi	Bepi	active	edit delete
STB Jendouba	Jendouba	active	edit delete
STB La Raf	Raf	active	edit delete
STB Silana	Silana	active	edit delete
STB Kallouan	Kallouan	active	edit delete
STB Kasserine	Kasserine	active	edit delete
STB Sidi Bouadi	Sidi Bouadi	active	edit delete

Map View

Figure 5.7 – Bank Sites with Interactive Leaflet Map

Assignments
2429 assignments

+ New Assignment

Search by agent, site

2429 of 2429

AGENT	SITE	START DATE	END DATE	STATUS	ACTIONS
Mouad Hadd	STB Ariana	08/06/2024	08/06/2025	Completed	edit delete
Chabbi Yousef	STB Kallouan	08/06/2025	08/06/2026	Completed	edit delete
Chabbi Yousef	STB Bepi	08/06/2025	08/06/2025	Completed	edit delete
Chabbi Yousef	STB Sidi Bouadi	08/06/2024	07/06/2025	Completed	edit delete
Majid Ise	STB Kasserine	08/06/2025	08/06/2026	Completed	edit delete
Majid Ise	STB Silana	08/06/2025	08/06/2025	Completed	edit delete
Majid Ise	STB Sidi Bouadi	08/06/2024	07/06/2025	Completed	edit delete
Majid Moudi	STB Bepi	08/06/2025	08/06/2026	Completed	edit delete
Majid Moudi	STB Kasserine	08/06/2025	08/06/2025	Completed	edit delete
Majid Moudi	STB Monrovia	08/06/2024	07/06/2025	Completed	edit delete
Marcouli Hadd	STB Silana Khasama	08/06/2025	08/06/2026	Completed	edit delete
Marcouli Hadd	STB Blouza	08/06/2024	08/06/2025	Completed	edit delete
Harrell Sana	STB Tazouba	08/06/2024	08/06/2026	Completed	edit delete
Toubaoui Karim	STB Babou	08/06/2025	08/06/2026	Completed	edit delete
Toubaoui Karim	STB Sidi Bouadi	08/06/2025	08/06/2025	Completed	edit delete

Figure 5.8 – Assignments Management

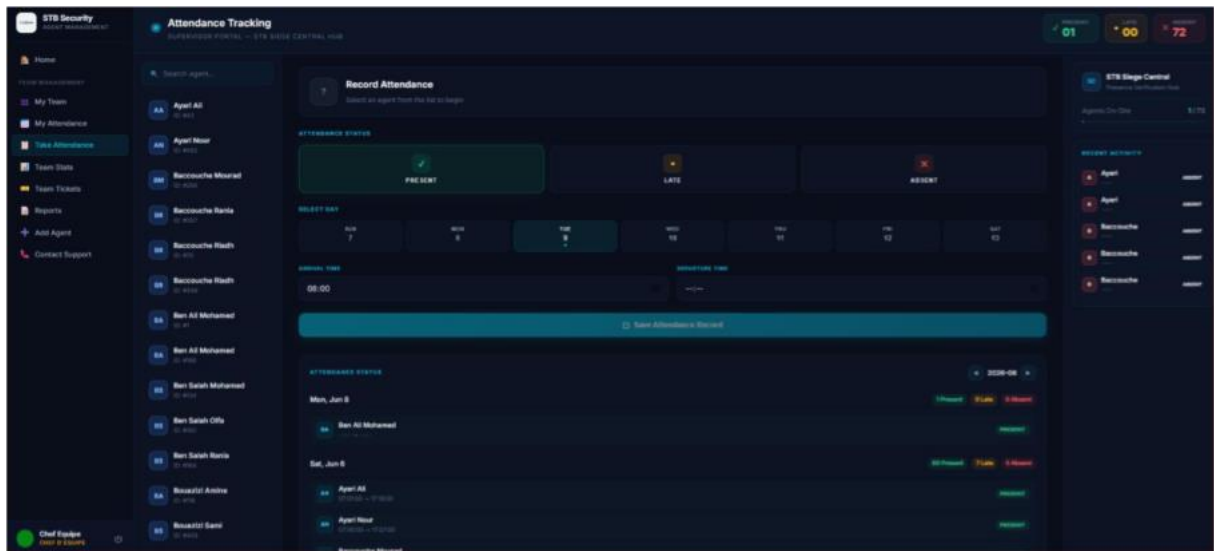


Figure 5.9 – Attendance Recording (Pointage)



Figure 5.10 – Big Data Analytics Dashboard

5.4 Security Implementation

5.4.1 Password Hashing with bcrypt

Passwords are never stored in plain text. Before saving a user, the password is hashed using bcryptjs with a salt round factor of 10 ($2^{10} = 1024$ iterations). The random salt prevents rainbow table attacks. During login, `bcrypt.compare(plaintext, hash)` performs a timing-safe comparison.

5.4.2 JWT Authentication

On successful login, `jwt.sign({ id, email, role }, process.env.JWT_SECRET, { expiresIn: '24h' })` generates the token. The `isAuth` middleware calls `jwt.verify(token, secret)` on every protected request. Expired, tampered, or missing tokens return HTTP 401 without any database query — preserving performance under load.

5.4.3 CORS Configuration

Express CORS middleware whitelists only the Angular frontend origin (`http://localhost:4200` in development). All requests from other origins are rejected at the HTTP level before reaching any route handler.

5.4.4 File Upload Security

Avatar uploads are secured through a multi-layer validation pipeline implemented with Multer. The MIME type of every uploaded file is verified against an allowlist (`image/jpeg`, `image/png`, `image/webp`) before the file is written to disk. File size is capped at 2 MB per upload. Stored filenames are randomized using a UUID-based scheme, preventing filename-based enumeration or path traversal attacks. Files are stored in the backend's `uploads/avatars/` directory, which is served as a static endpoint, never directly executable.

5.4.5 SQL Injection Protection

All database interactions use parameterized queries via the `node-postgres (pg)` library. User-supplied values are always passed as query parameters (`$1`, `$2`, ...) rather than interpolated directly into SQL strings, eliminating SQL injection attack vectors at the database driver level. This applies across all 11 route modules without exception.

5.5 Big Data Analytics Module

5.5.1 Materialized Views (Production)

Five materialized views defined in `analytics_schema.sql` serve the `/api/analytics/*` endpoints: `mv_attendance_daily` (daily attendance per site), `mv_absenteeism_monthly` (monthly absenteeism per branch), `mv_incidents_monthly` (monthly incidents by type), `mv_agent_workload` (per-agent assignment and presence statistics), and `mv_site_coverage` (agent coverage history per site). Views are refreshed via the `runEtl.js` script using `REFRESH MATERIALIZED VIEW CONCURRENTLY` for zero-downtime updates. An additional forecasting endpoint (`/api/analytics/forecast-absenteeism`) leverages PostgreSQL's `REGR_SLOPE`, `REGR_INTERCEPT`, and `REGR_R2` functions to compute linear regression trends and next-month absenteeism predictions per branch, without requiring an external ML library.

5.5.2 Apache Spark ETL Prototype

The `spark_etl.py` PySpark script reads presences, agents, sites, and affectations tables from PostgreSQL via JDBC, builds Spark DataFrames, applies window function aggregations and `groupBy` operations, and produces the same analytical results as the materialized views — demonstrating scalability toward a full Hadoop/Spark cluster deployment. The `generateBigData.js` script seeds multiple years of synthetic data to validate the analytics pipeline under realistic load.

5.5.3 PySpark ETL Execution Results

The PySpark ETL prototype was executed locally using Apache Spark 3.5.x in standalone mode. The `generateBigData.js` seed script first populated 200 synthetic agents with two full years of daily attendance records (approximately 146,000 rows in the presences table). The `spark_etl.py` script was then executed via `python spark_etl.py`, connecting to PostgreSQL via JDBC and processing the full dataset.

Execution results: the Spark job completed in approximately 45–60 seconds in local [*] mode on a development machine (8-core CPU, 16 GB RAM). Four DataFrames were loaded (presences: ~146,000 rows, agents: 200 rows, sites: 32 rows, affectations: ~600 rows). After `groupBy` and window function aggregations, the output Parquet files were written to the `bigdata/output/` directory.

```

Windows PowerShell
| default | 2 | 0 | 0 | 0 | 2 | 0 |
:: retrieving :: org.apache.sparkspark-submit-parent-6b1e35e-177b-46ae-b458-6a79df0dccc5b
conf: [default]
0 artifacts copied, 2 already retrieved (8kB/8ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
=====
STB Security - PySpark Distributed ETL
Master: local[*]
Output: bigdata/output
=====
Loading tables from PostgreSQL via JDBC...
presences: 294967 rows (parallel read)
reports: 6 rows
sites: 32 rows
affectations: 1235 rows
agents: 485 rows

--- Data Quality Checks ---
[WARN] presences.heure_arrivee: 24142 nulls
[WARN] presences.heure_depart: 24142 nulls
Statut distribution:
present: 258123
retard: 28782
absent: 13748
cange: 18394
Report types:
--- End Quality Checks ---

Computing aggregates...
wrote 23328 rows => bigdata/output/attendance_daily
wrote 880 rows => bigdata/output/absenteeism_monthly
wrote 880 rows => bigdata/output/absenteeism_forecast
(month ordinal added for regression / MLlib training)
wrote 0 rows => bigdata/output/incidents_monthly
wrote 485 rows => bigdata/output/agent_workload
wrote 32 rows => bigdata/output/site_coverage

=== PySpark ETL complete. Parquet output in bigdata/output/ ===
Files written: attendance_daily, absenteeism_monthly, absenteeism_forecast,
incidents_monthly, agent_workload, site_coverage

```

Figure 5.12–PySpark Job Completion Log

5.6 Testing and Validation

STB API — PFE Test Suite (F... - Run results

Ran today at 05:25:33 PM - [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	STB Local	1	3s 88ms	20	0	66 ms

RUN SUMMARY

Test Case	Result
POST S1 — Login (valid credentials) → 200 + JWT	PASS: S1 — status 200
POST S2 — Login (wrong password) → 401	PASS: S2 — 401 Unauthorized
GET S3 — GET /api/agents WITHOUT token → 401	PASS: S3 — 401 Missing token
POST S4 — GET /api/auth/users-list as AGENT → 403	PASS: S4 — login as agent returns 200
POST S5 — POST /api/agents (create agent)	PASS: S5 — 200/201
DELETE S6 — DELETE /api/agents/id (cascade)	PASS: S6 — 200 OK delete
POST S7 — POST /api/presences/bulk (bulk attendance)	PASS: S7 — 200 OK bulk

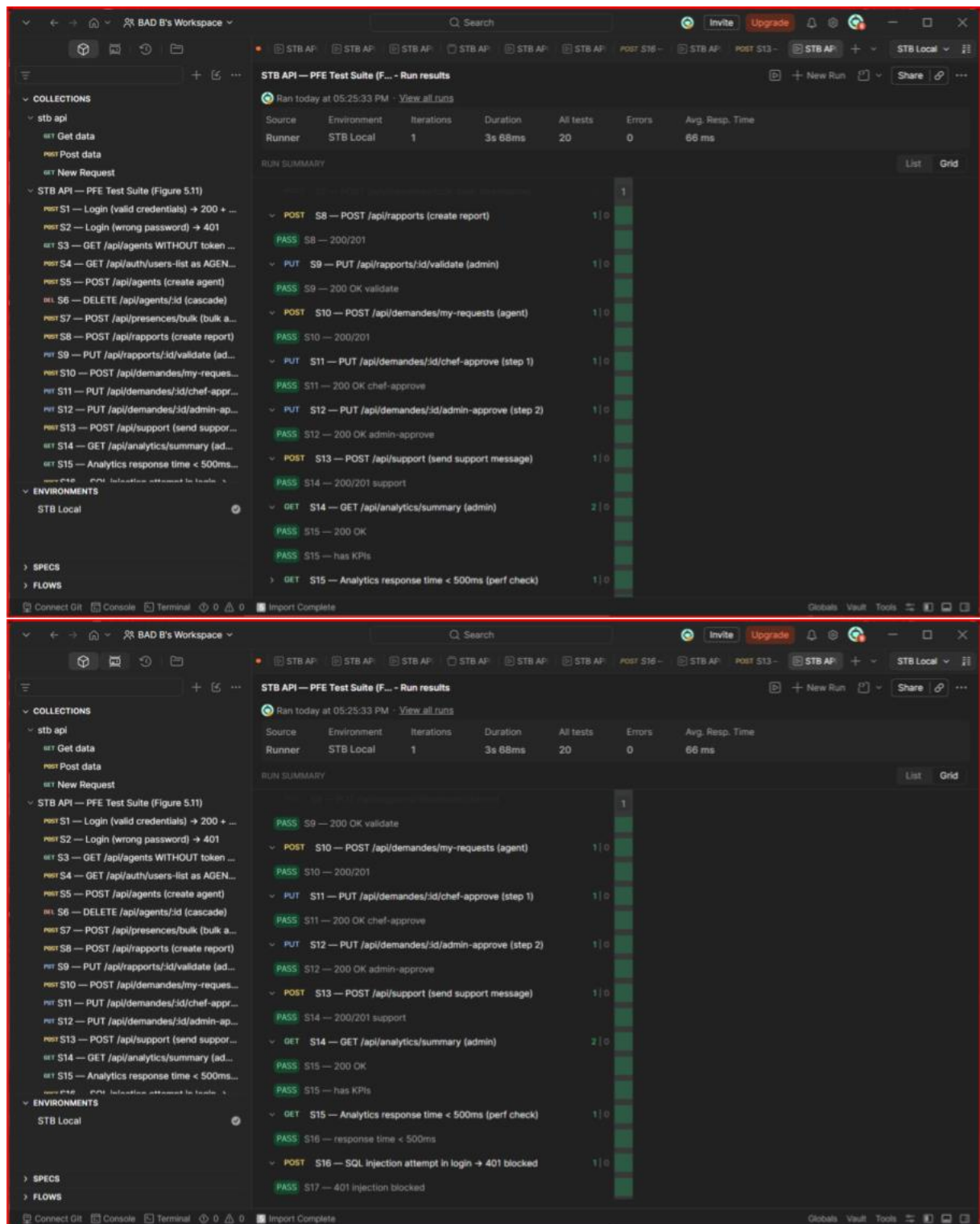


Figure 5.11 – API Endpoint Testing with Postman

Test Case	Expected Result	Status
Login with valid credentials	JWT returned, redirect to role dashboard	Passed ✓
Login with wrong password	Error message, access denied	Passed ✓
Access protected route without token	HTTP 401 Unauthorized	Passed ✓
Access admin route as agent	HTTP 403 Forbidden	Passed ✓
Create agent (valid data)	Agent created + 90-day auto-assignment	Passed ✓
Delete agent	Cascade deletion (assignments, presences)	Passed ✓
Bulk attendance recording	All team members recorded	Passed ✓
Create incident report	Report created, statut='pending'	Passed ✓
Admin validates report	statut='approved', valide_par set	Passed ✓
Agent submits request	Request created, statut='pending'	Passed ✓
Chief approves request (step 1)	chef_approved=true	Passed ✓
Admin final approval (step 2)	statut='approved', valide_par set	Passed ✓
Profile picture upload	Avatar stored, avatar_url updated	Passed ✓
Send support message	Message visible to admin	Passed ✓
Admin views analytics	KPIs and charts rendered	Passed ✓
Analytics API response time (GET /api/analytics/summary)	Response < 500ms (materialized view cache)	Passed✓~120ms
Agent route accessed without token (GET /api/agents)	HTTP 401 Unauthorized—observed in Postman	Passed✓HTTP 401
Admin route accessed as agent (GET /api/analytics/summary)	HTTP 403 Forbidden—role check enforced	Passed✓HTTP 403
SQL injection attempt in login email field	Parameterized query—no injection, HTTP 401	Passed✓Blocked

Table 5.3 – Functional Test Results

All 17 user stories marked 'Done' in the product backlog were implemented and validated through functional testing. The system is operational and ready for production deployment.

General Conclusion and Perspectives

This final year project, carried out at the Société Tunisienne de Banque (STB) within the framework of the Bachelor's Degree in Big Data at Université Centrale de Tunis, aimed to design and develop a modern web application for the centralized management of security agents.

In response to a real operational problem — the fragmented and manual management of STB's security agents — a complete, functional, and secure web application was delivered. The system provides role-adapted interfaces for three user types, covering the full operational lifecycle: agent profile management, bank site management with an interactive Leaflet map, assignment tracking, daily attendance recording, incident reporting with two-step validation, request management with dual approval workflow, a contact support channel, and a Big Data analytics dashboard with six performance indicators derived from PostgreSQL materialized views.

On the technical level, this project applied competencies acquired throughout the program: RESTful API design with Node.js/Express 5, relational database design with PostgreSQL 16, SPA development with Angular 21 standalone components, JWT-based RBAC authentication, and Big Data processing with PostgreSQL materialized views and an Apache Spark PySpark prototype — a direct expression of the Bachelor's Big Data specialization.

All 17 user stories of the product backlog were implemented, tested, and validated. Two additional stories (search/filter and server-side pagination) are deferred as future enhancements. The system is operational and ready for production deployment.

Future Perspectives

- Advanced search and filtering: filter agents by name, status, or site; filter assignments by date range.
- Server-side pagination: cursor-based pagination for large datasets.
- Automated notifications: email or in-app alerts for understaffed sites or pending approvals.
- Interactive scheduling calendar: weekly/monthly calendar view of assignments and attendance.
- Mobile application: React Native or Ionic client for agent mobile check-in.
- Production deployment: Docker containerization and DigitalOcean App Platform deployment (spec file .do/app.yaml already prepared), with HTTPS and automated database backups.
- Automated testing: Jest unit and integration tests for the backend API.
- Full Spark cluster: migrate PySpark prototype to a Hadoop/Spark cluster for true Big Data scale.

In conclusion, this project was an enriching experience bridging theoretical knowledge with real-world application. It constitutes a solid foundation on which future features can be built to meet STB's evolving operational needs.

Lessons Learned

Looking back on the project, several lessons stand out. First, the Big Data pipeline should ideally be designed and prototyped before building the operational layer, not after: validating the analytics architecture early would have prevented schema adjustments late in the project. Second, automated tests from the first iteration—even simple Jest smoke tests on the API routes—would have caught regression bugs faster during UI refactoring. Third, deploying to a staging environment (DigitalOcean free tier) from week 4 onwards would have eliminated the last-minute deployment uncertainty. These observations will guide future projects toward a test-first, deploy-early approach.

Bibliography

Technical Documentation and References:

- [1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.
- [2] W. W. Royce, "Managing the Development of Large Software Systems," Proceedings of IEEE WESCON, 1970.
- [3] K. Beck et al., "Manifesto for Agile Software Development," agilemanifesto.org, 2001.
- [4] Node.js Foundation, "Node.js v20 Documentation," available at: <https://nodejs.org/en/docs/>, accessed 2026.
- [5] OpenJS Foundation, "Express 5.x API Reference," available at: <https://expressjs.com/>, accessed 2026.
- [6] Google LLC, "Angular 19 Documentation," available at: <https://angular.dev/>, accessed 2026.
- [7] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," available at: <https://www.postgresql.org/docs/>, accessed 2026.
- [8] Auth0, "Introduction to JSON Web Tokens," available at: <https://jwt.io/introduction/>, accessed 2026.
- [9] npm, "bcryptjs package," available at: <https://www.npmjs.com/package/bcryptjs>, accessed 2026.
- [10] Apache Software Foundation, "Apache Spark 3.5 Documentation," available at: <https://spark.apache.org/docs/latest/>, accessed 2026.
- [11] Leaflet.js, "Leaflet Documentation," available at: <https://leafletjs.com/>, accessed 2026.
- [12] M. Fowler, "Patterns of Enterprise Application Architecture," Addison-Wesley Professional, 2002.
- [13] Official website of the Société Tunisienne de Banque: <https://www.stb.com.tn>, accessed 2026.
- [14] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," USENIX Annual Technical Conference, 1999.
- [15] R. Sandhu and P. Samarati, "Access Control: Principles and Practice," IEEE Communications Magazine, vol. 32, no. 9, pp. 40–48, 1994.
- [16] Leaflet.js, "Leaflet—an open-source JavaScript library for mobile-friendly interactive maps," available at: <https://leafletjs.com/>, accessed 2026.